

Capítulo IV: Solution Software Design.

4.1. Strategic-Level Domain-Driven Design.

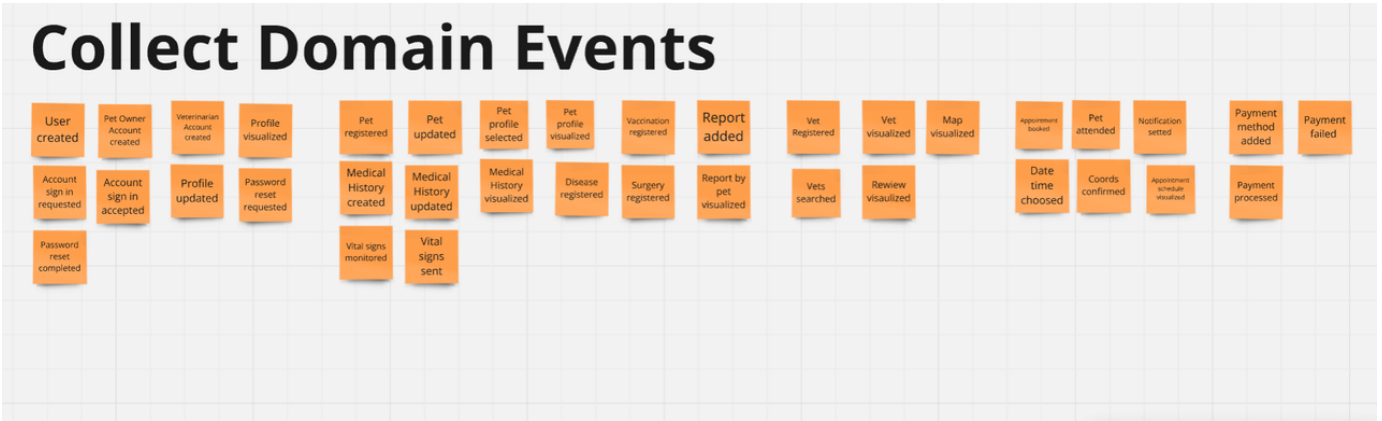
En esta sección, explicaremos el proceso de toma de decisiones estratégicas de diseño de software aplicando Domain-Driven Design (DDD). Nos enfocamos en identificar subdominios críticos y contextos delimitados, alineando el diseño con los objetivos del negocio. Mediante técnicas colaborativas como EventStorming, modelamos flujos de mensajes y definimos contextos adecuados y sus interrelaciones. Luego, Context Mapping ayudó a establecer relaciones y patrones de integración, formando una base sólida para la arquitectura. Se detallan aquí los pasos clave, desde el descubrimiento inicial hasta la arquitectura final, fundamentados en los principios de DDD.

4.1.1. EventStorming.

Se llevó a cabo un proceso de Event Storming para identificar los Bounded Contexts de nuestro sistema. Durante este proceso, se siguieron los pasos que se describen a continuación:

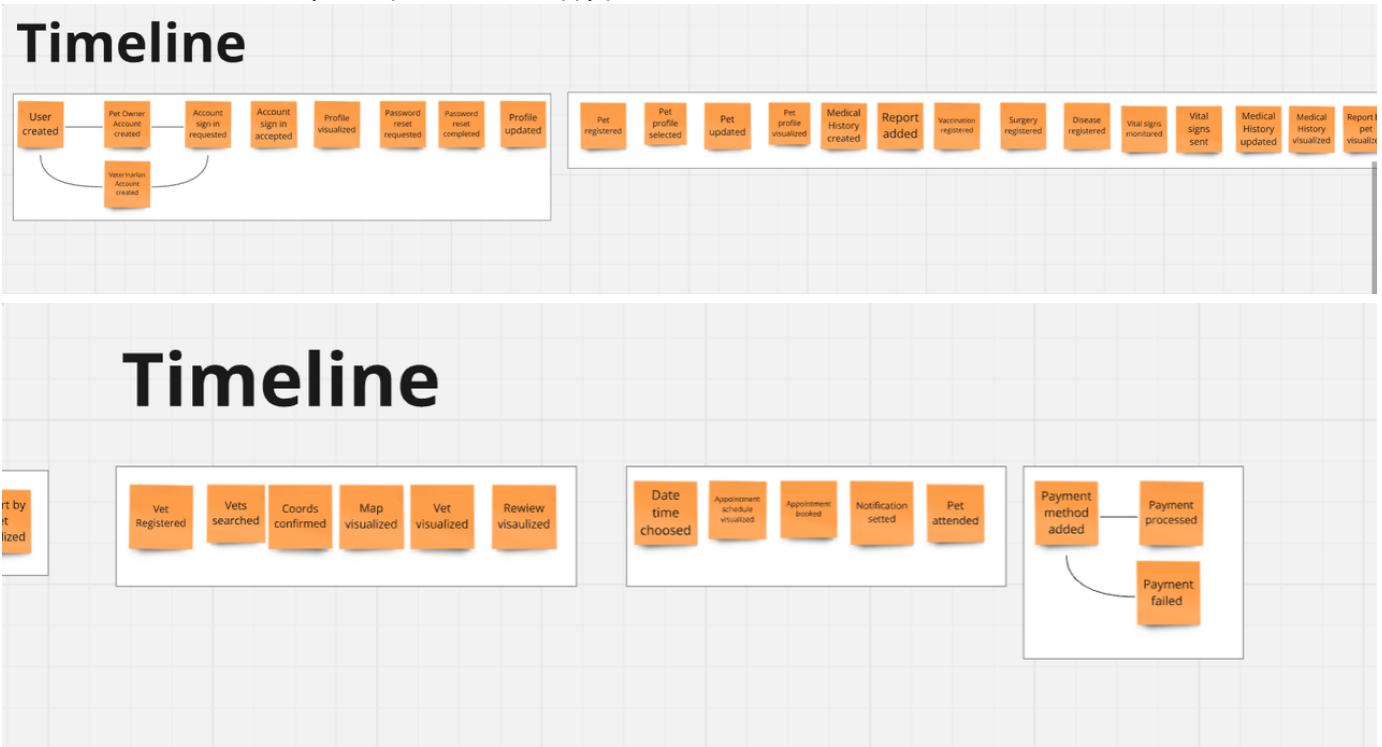
Collect Domain Events

Se plantean eventos importantes de todos los grupos funcionales en tiempo pasado y nomenclatura en inglés.



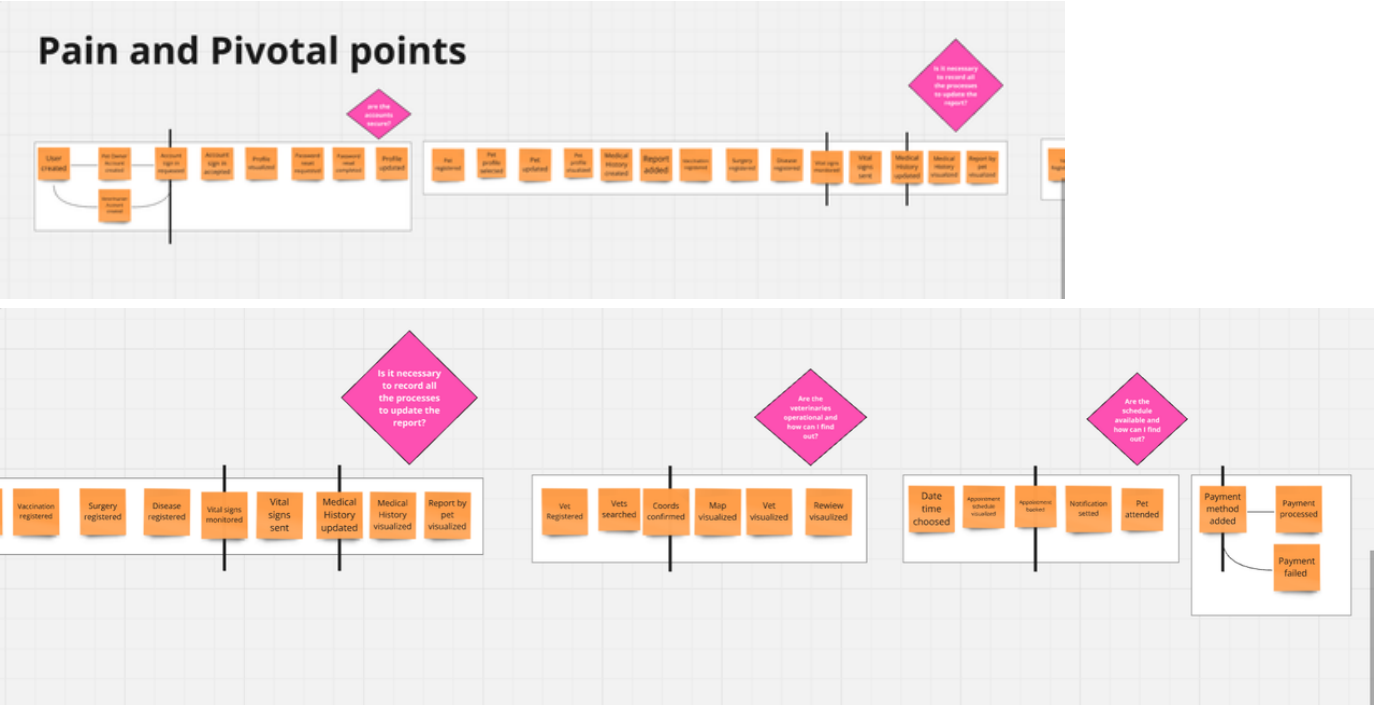
Timeline

Se ordenan todos los eventos y se empieza con un "happy path" es decir, eventos exitosos.

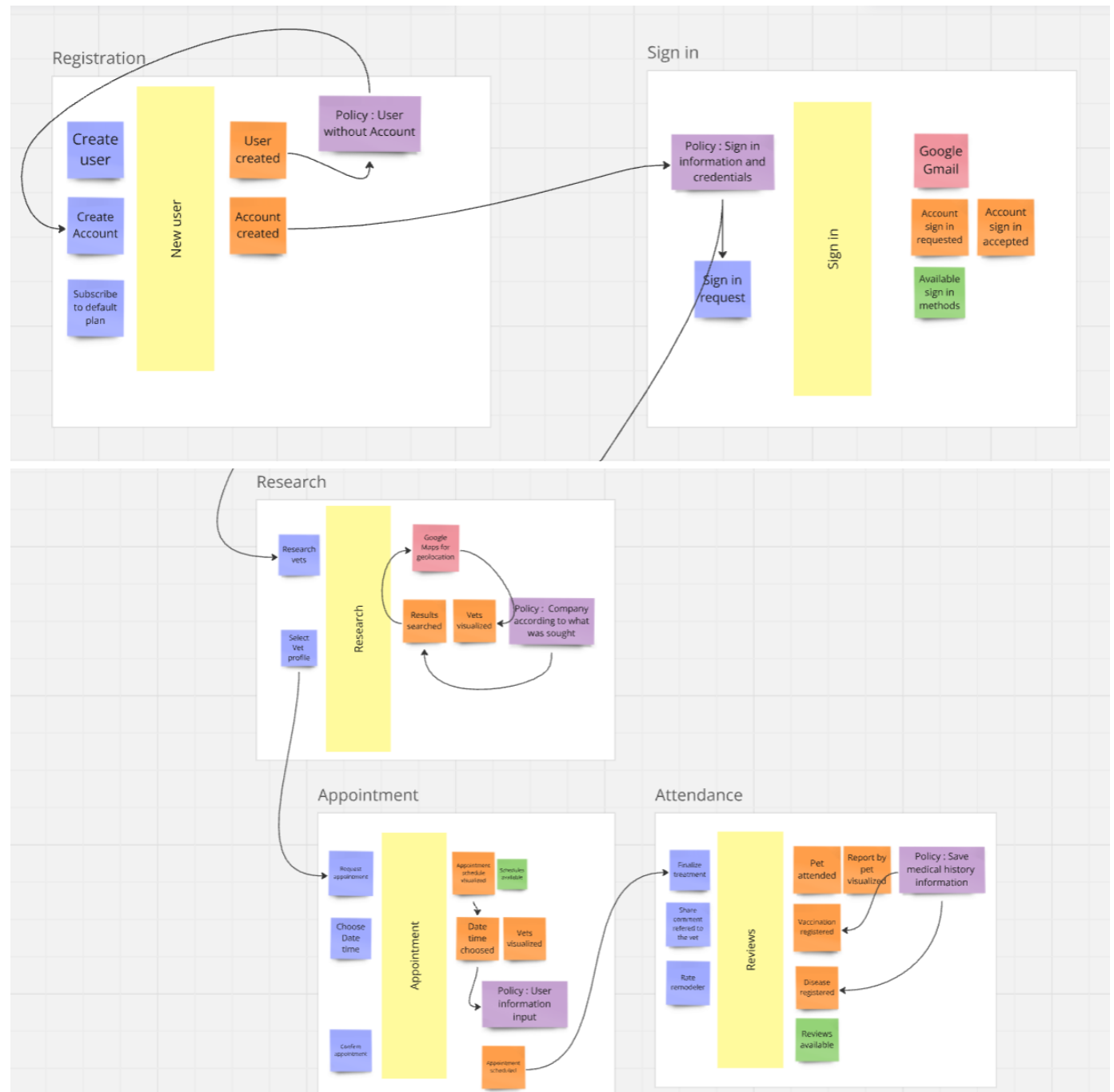


Pain and Pivotal Points

En este paso se resaltan con un diamante los eventos por aclarar o que requieren de más conocimientos de especialistas. Por otro lado, los pivotal points son puntos de cambios que se marcan con una barra vertical. Por otro lado, los pivotal points son puntos de cambios que se marcan con una barra vertical.



4.1.1.1 Candidate Context Discovery

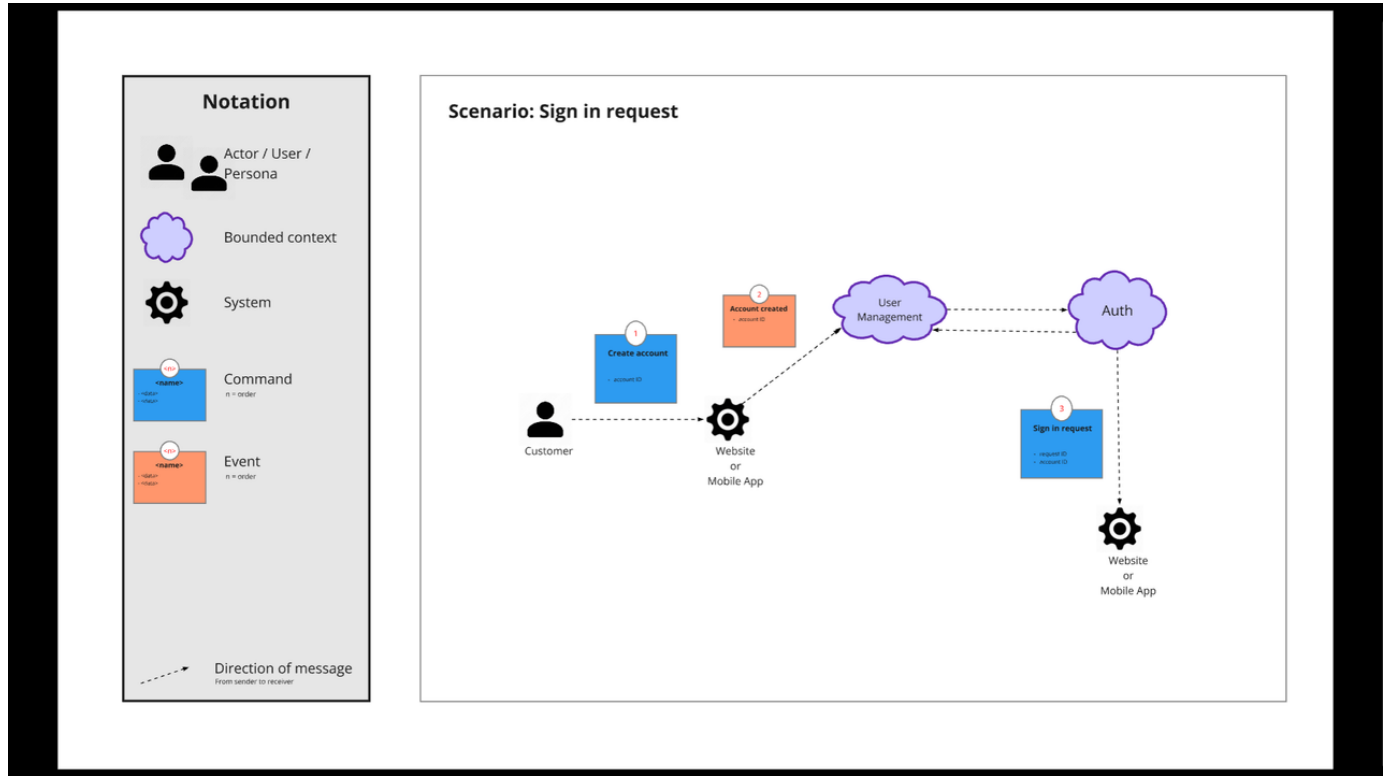


4.1.1.2 Domain Message Flows Modeling.

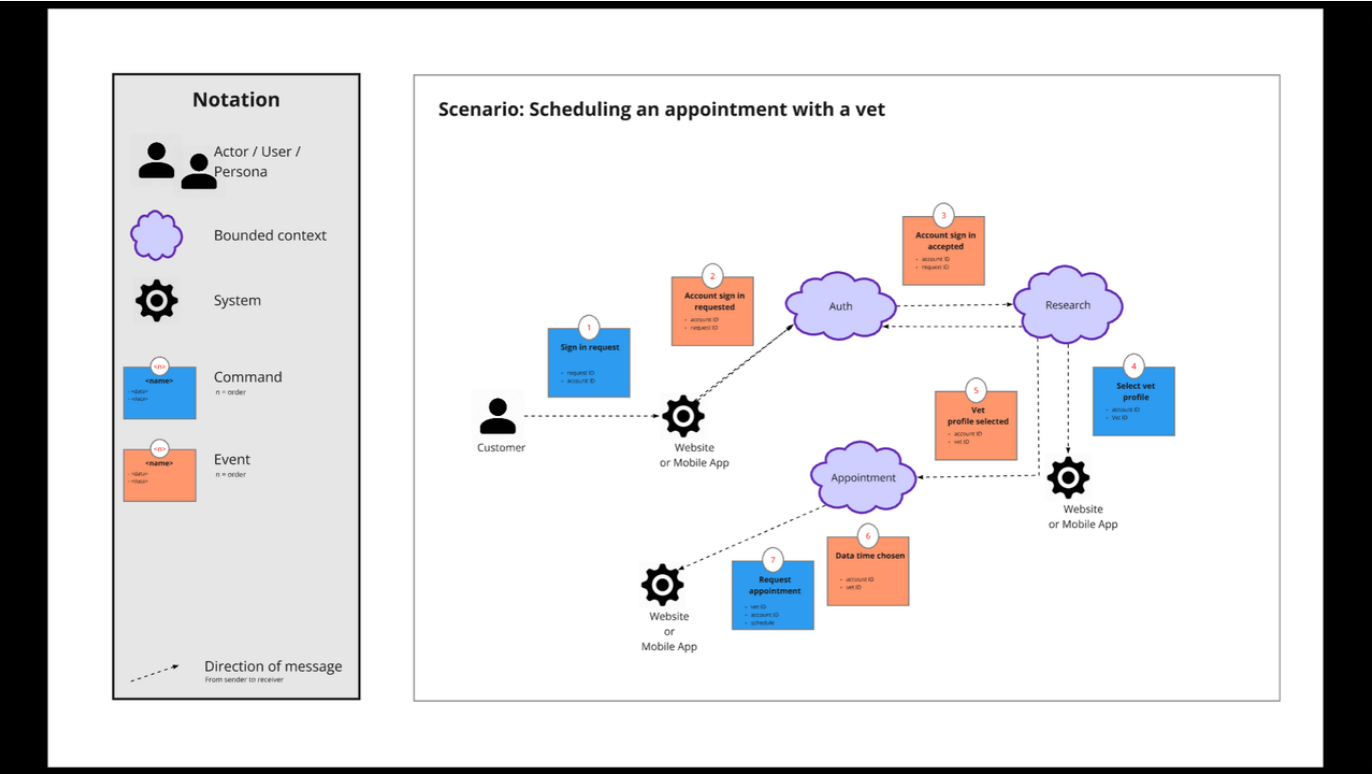
Dado el diagrama de eventos de EventStorming que ya se ha presentado, el siguiente paso es modelar los flujos de mensajes. Para esto, podemos identificar las interacciones clave que ocurren en el sistema, y cómo estos mensajes desencadenan acciones o actualizaciones en otros contextos.

Solicitud de inicio de sesión: En este escenario, un actor (Customer) crea una cuenta en nuestro sistema y, de inmediato, se suscribe al plan predeterminado para validar sus credenciales, gracias a una política establecida. Este evento se registra en nuestro backend, lo que confirma la suscripción. Una vez verificada la suscripción dentro del bounded context de Access, al cliente se le permitirá acceder

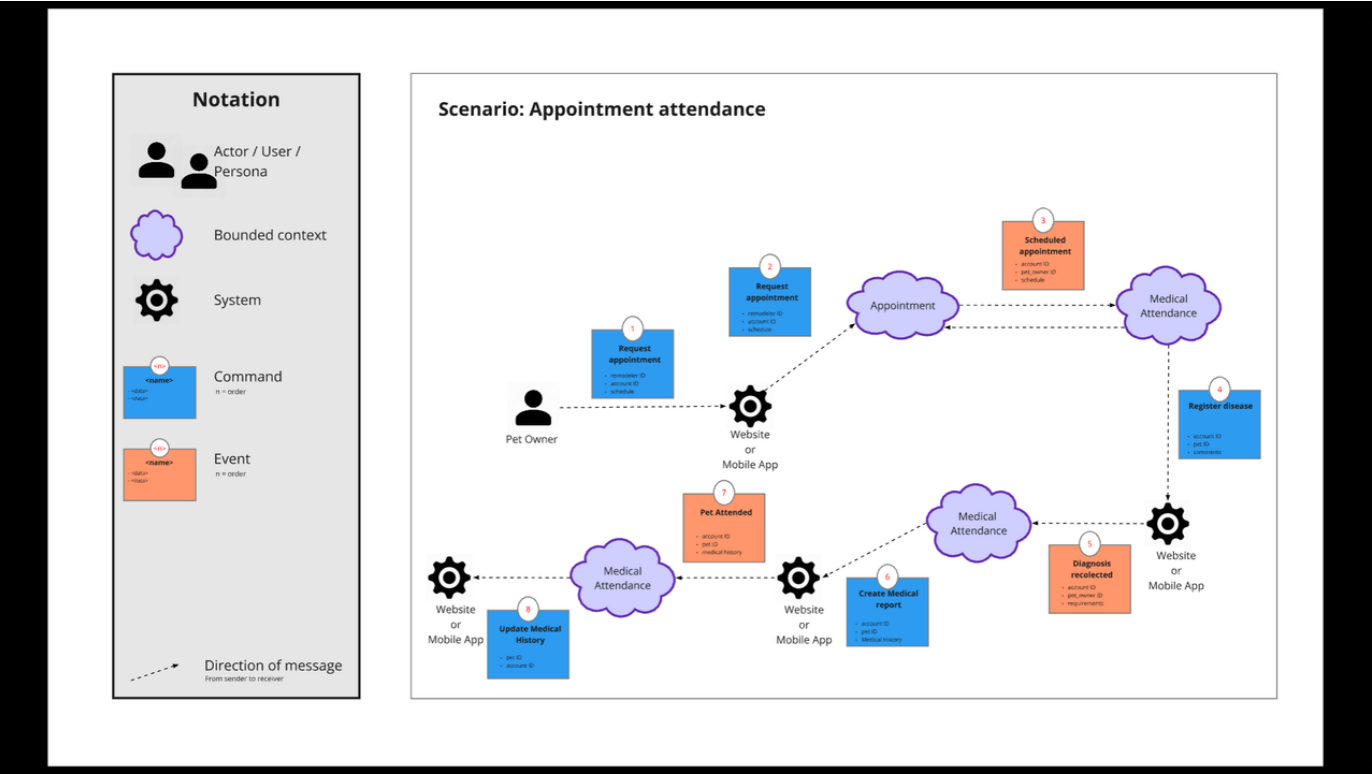
al comando de solicitud de inicio de sesión (Sign in request).



solicitar una cita utilizando el comando (*Request appointment*).



Manejar el historial médico de la mascota: Para gestionar el historial médico, el actor (Customer) solicita una cita con un veterinario. Luego, se le pedirá información relevante sobre diagnósticos de enfermedades y actualizaciones de reportes. Este comando se ejecuta desde nuestro frontend mediante (*Collect medical information*). Si la información se completa, se envía a la plataforma a través de (*Send medical information to platform*). Finalmente, una vez que toda la información esté en nuestro backend, el veterinario puede actualizar el historial médico de la mascota usando el comando (*Update medical history*).



4.1.1.3 Bounded Context Canvases.

Se crearon lienzos de Bounded Context para cada uno de los contextos identificados en el proceso de EventStorming. Estos lienzos ayudan a definir los límites de cada contexto, sus responsabilidades y las interacciones con otros contextos.

Name Auth	Model Traits Draft, execute, audit, gateway.... 												
Description Summary of purposes and responsibilities This context exists to verify credentials of an account and allow a user sign in to the platform	Messages Consumed and Produced <table><tr><td>Messages Consumed</td><td>Messages Produced</td></tr><tr><td> Commands Handled Events Handled Queries Handled</td><td> Commands Sent Events Published Queries Invoked</td></tr></table>	Messages Consumed	Messages Produced	 Commands Handled Events Handled Queries Handled	 Commands Sent Events Published Queries Invoked								
Messages Consumed	Messages Produced												
 Commands Handled Events Handled Queries Handled	 Commands Sent Events Published Queries Invoked												
Strategic Classification Domain - generic Business Model - compliance Evolution - commodity													
Business Decisions Key business rules, policies, and decisions 	Dependencies and Relationships <table><tr><td>Message Suppliers</td><td>Message Consumers</td></tr><tr><td><table><tr><th>Name</th><th>Relationship</th></tr><tr><td>Sign in request</td><td>I can request to sign in into my account providing my personal account information</td></tr></table></td><td><table><tr><th>Name</th><th>Relationship</th></tr><tr><td></td><td></td></tr></table></td></tr></table>	Message Suppliers	Message Consumers	<table><tr><th>Name</th><th>Relationship</th></tr><tr><td>Sign in request</td><td>I can request to sign in into my account providing my personal account information</td></tr></table>	Name	Relationship	Sign in request	I can request to sign in into my account providing my personal account information	<table><tr><th>Name</th><th>Relationship</th></tr><tr><td></td><td></td></tr></table>	Name	Relationship		
Message Suppliers	Message Consumers												
<table><tr><th>Name</th><th>Relationship</th></tr><tr><td>Sign in request</td><td>I can request to sign in into my account providing my personal account information</td></tr></table>	Name	Relationship	Sign in request	I can request to sign in into my account providing my personal account information	<table><tr><th>Name</th><th>Relationship</th></tr><tr><td></td><td></td></tr></table>	Name	Relationship						
Name	Relationship												
Sign in request	I can request to sign in into my account providing my personal account information												
Name	Relationship												
Ubiquitous Language Key domain terminology 													

Name Research and scheduling	Model Traits Draft, execute, audit, gateway.... 																
Description Summary of purposes and responsibilities This context exists to facilitate the search and management of appointments between Pet owner with a Vet. The benefit would be the increase in services carried out on the platform.	Messages Consumed and Produced <table><tr><td>Messages Consumed</td><td>Messages Produced</td></tr><tr><td> Commands Handled Events Handled Queries Handled</td><td> Commands Sent Events Published Queries Invoked</td></tr></table>	Messages Consumed	Messages Produced	 Commands Handled Events Handled Queries Handled	 Commands Sent Events Published Queries Invoked												
Messages Consumed	Messages Produced																
 Commands Handled Events Handled Queries Handled	 Commands Sent Events Published Queries Invoked																
Strategic Classification Domain - supporting Business Model - engagement Evolution - commodity																	
Business Decisions Key business rules, policies, and decisions 	Dependencies and Relationships <table><tr><td>Message Suppliers</td><td>Message Consumers</td></tr><tr><td><table><tr><th>Name</th><th>Relationship</th></tr><tr><td>Access</td><td>Provide the login information</td></tr><tr><td>Review</td><td>Provide the login information</td></tr></table></td><td><table><tr><th>Name</th><th>Relationship</th></tr><tr><td>Notification</td><td>Consume notification information</td></tr><tr><td></td><td>Consumes the information of the requirements</td></tr></table></td></tr></table>	Message Suppliers	Message Consumers	<table><tr><th>Name</th><th>Relationship</th></tr><tr><td>Access</td><td>Provide the login information</td></tr><tr><td>Review</td><td>Provide the login information</td></tr></table>	Name	Relationship	Access	Provide the login information	Review	Provide the login information	<table><tr><th>Name</th><th>Relationship</th></tr><tr><td>Notification</td><td>Consume notification information</td></tr><tr><td></td><td>Consumes the information of the requirements</td></tr></table>	Name	Relationship	Notification	Consume notification information		Consumes the information of the requirements
Message Suppliers	Message Consumers																
<table><tr><th>Name</th><th>Relationship</th></tr><tr><td>Access</td><td>Provide the login information</td></tr><tr><td>Review</td><td>Provide the login information</td></tr></table>	Name	Relationship	Access	Provide the login information	Review	Provide the login information	<table><tr><th>Name</th><th>Relationship</th></tr><tr><td>Notification</td><td>Consume notification information</td></tr><tr><td></td><td>Consumes the information of the requirements</td></tr></table>	Name	Relationship	Notification	Consume notification information		Consumes the information of the requirements				
Name	Relationship																
Access	Provide the login information																
Review	Provide the login information																
Name	Relationship																
Notification	Consume notification information																
	Consumes the information of the requirements																
Ubiquitous Language Key domain terminology 																	

Name Medical attendance	Model Traits Draft, execute, audit, gateway..... <Execue>
Description Summary of purposes and responsibilities This context exists so that the veterinarians can manage the medical history of the pets	Messages Consumed and Produced Messages Consumed <Command> Comm Hand <Event> Even Hand <Query> Queries Handled Messages Produced <Command> Com S <Event> Pet attended <Query> Queries Invoked
Strategic Classification Domain - supporting Business Model - engagement Evolution - product	
Business Decisions Key business rules, policies, and decisions Policy : Pet attendance <Decision>	Dependencies and Relationships Message Suppliers Name Medical Attendance Relationship Provides proposal and pet information. Message Consumers Name Review Relationship Consume finished Name Notification Relationship Consume notification information
Ubiquitous Language Key domain terminology <Domain Term> <definition>	

Name Reviewing	Model Traits Draft, execute, audit, gateway..... <Trait>
Description Summary of purposes and responsibilities This context exists so that the pet owner can qualify the work of the remodeler.	Messages Consumed and Produced Messages Consumed <Command> Com Ha <Event> Ev Ha <Query> Queries Handled Messages Produced <Command> Co ds <Event> Vet rated <Query> Reviews available
Strategic Classification Domain - generic Business Model - compliance Evolution - commodity	
Business Decisions Key business rules, policies, and decisions <Decision> Policy : Save reviews	Dependencies and Relationships Message Suppliers Name Pet Attendance Relationship Message Consumers Name Research and Appointment Relationship
Ubiquitous Language Key domain terminology <Pet owner> <Person who ask for medical attention>	

Name Monitoring	Model Traits Draft, execute, audit, gateway.... 
Description Summary of purposes and responsibilities This context exists so that the vets and pet owners can monitorate the vital signs and geolocation of the pet	Messages Consumed and Produced Messages Consumed 1 <Command> Collar and Handled 5 <Event> Signals read 15 <Query> Queries Handled Messages Produced 1 <Command> Send signals man Sent 5 <Event> Signals sent vents Published 15 <Query> Queries Invoked
Strategic Classification Domain - generic Business Model - compliance Evolution - commodity	Dependencies and Relationships Message Suppliers Name Monitoring signals Relationship Provides the signals of the pet Message Consumers Name Getting signals Relationship Data is retrieved to the pet owners and vets
Ubiquitous Language Key domain terminology <Pet owner> <person who wants to schedule an appointment> <Vet> <Property vet clinic> <Collar inteligente> <Iot device for pets>	

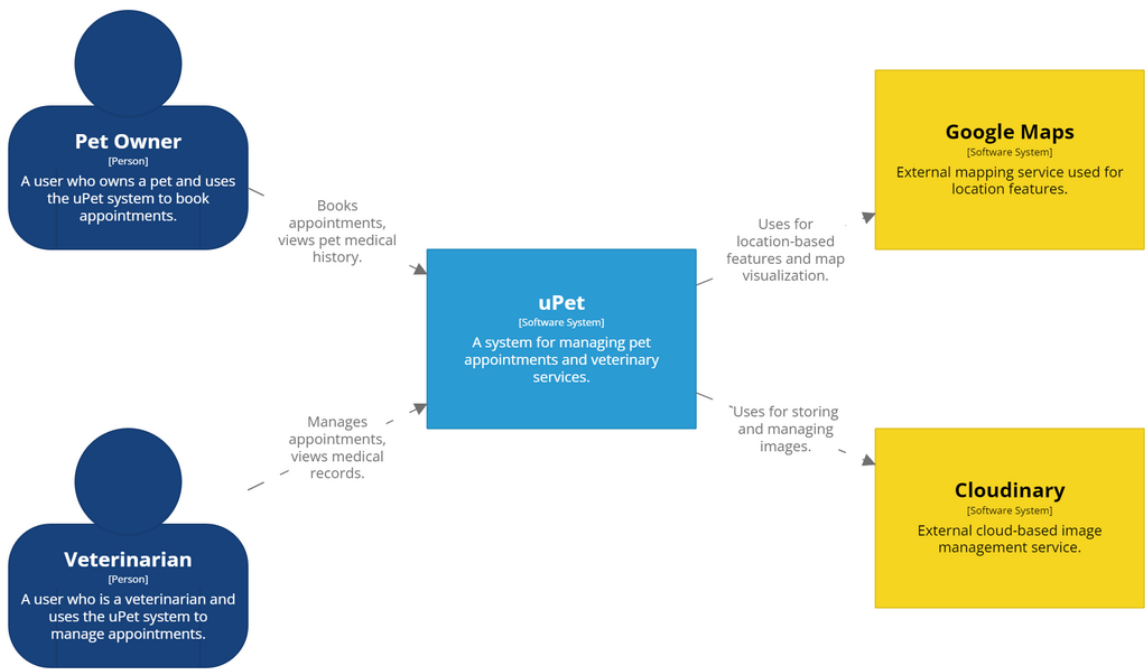
4.1.2. Context Mapping.

4.1.3. Software Architecture.

En esta sección, se describe la Arquitectura de Software de la solución utilizando el **C4 Model** para su representación visual, a través de la herramienta Structurizr. Se introducirá la estructura general del sistema, comenzando por una vista de alto nivel (Context Level Diagram) y detallando las interacciones y componentes clave (Container Level Diagrams), proporcionando así una visión clara y comprensible de la arquitectura propuesta.

4.1.3.2. Software Architecture Context Level Diagrams.

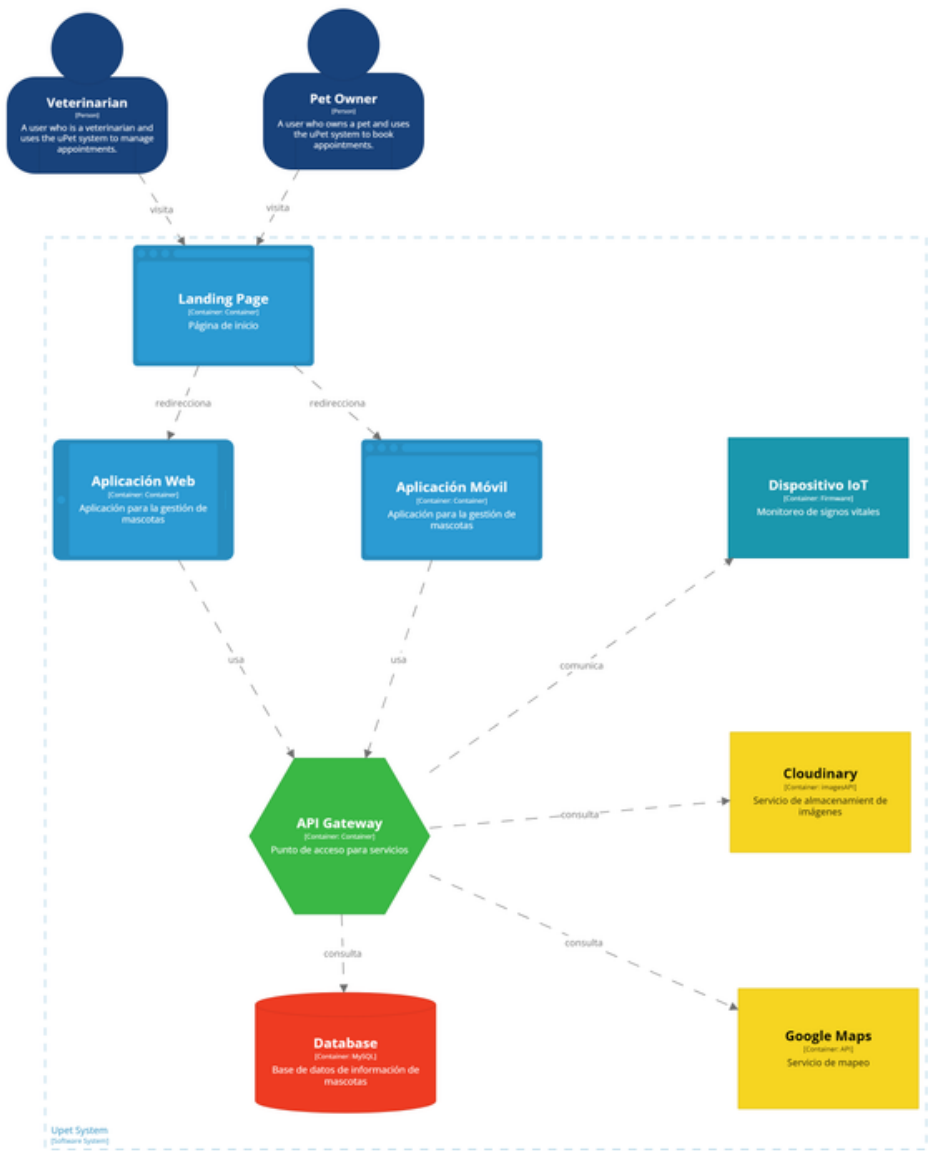
A continuación mostramos el diagrama de contexto de la arquitectura de software de la solución propuesta. En este diagrama se presentan los actores externos que interactúan con el sistema y los sistemas externos con los que se comunica.



[System Context] uPet
domingo, 20 de octubre de 2024, 12:27 hora estándar de Perú

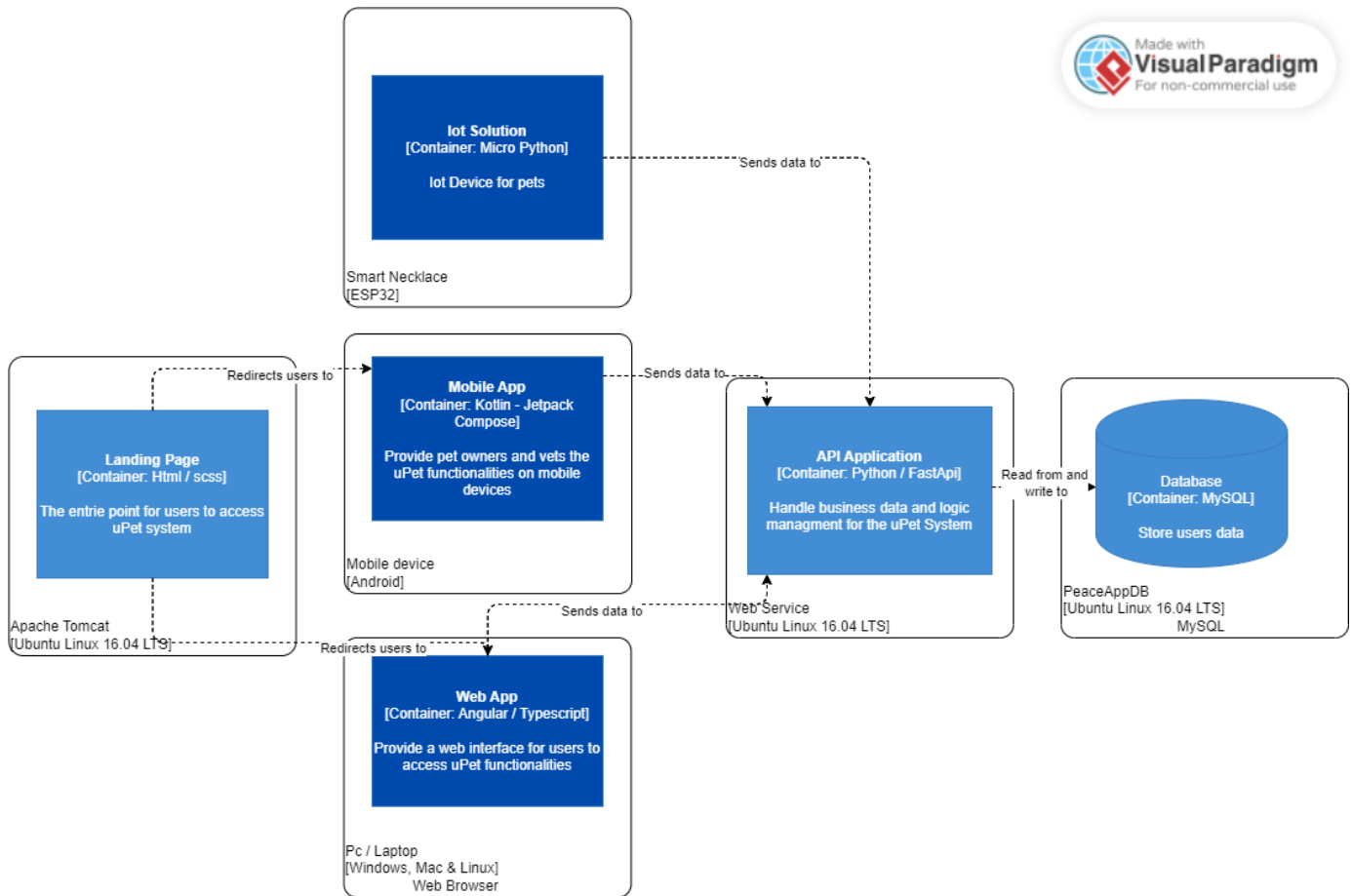
4.1.3.3. Software Architecture Container Level Diagrams.

En esta sección se presenta el diagrama de contenedores de la solución propuesta. Este diagrama detalla los contenedores de software y sus interrelaciones, proporcionando una visión general de la estructura interna del sistema.



[Container] Upet System
Diagrama de contenedores del sistema Upet
domingo, 20 de octubre de 2024, 13:15 hora estándar de Perú

4.1.3.4. Software Architecture Deployment Diagrams.



4.2. Tactical-Level Domain-Driven Design

4.2.1. Bounded Context: Identity and Access Context

- [Domain Layer](#)
- [Interface Layer](#)
- [Application Layer](#)
- [Infrastructure Layer](#)
- [Bounded Context Software Architecture Component Level Diagrams](#)
- [Bounded Context Software Architecture Code Level Diagrams](#)

4.2.1.1. Domain Layer

Models

Clase	Descripción
User	Representa la entidad de usuario con atributos como <code>id</code> , <code>name</code> , <code>email</code> , <code>password</code> , <code>userType</code> , etc. Relacionado con <code>PetOwner</code> y <code>Veterinarian</code> .
PetOwner	Representa la entidad de propietario de mascotas con atributos como <code>id</code> , <code>userId</code> , <code>numberPhone</code> , <code>location</code> , <code>subscriptionType</code> , etc. Relacionado con <code>User</code> , <code>Pet</code> , y <code>Review</code> .
Veterinarian	Representa la entidad de veterinario con atributos como <code>id</code> , <code>user_id</code> , <code>description</code> , <code>experience</code> , <code>clinic_id</code> , etc. Relacionado con <code>User</code> , <code>VeterinaryClinic</code> , <code>Availability</code> , <code>Appointment</code> , y <code>Review</code> .

Enums

Enum	Descripción
UserType	Enum para los tipos de usuarios: <code>Vet</code> , <code>Owner</code> .
SubscriptionType	Enum para los tipos de suscripciones: <code>Basic</code> , <code>Advanced</code> , <code>Pro</code> .

Validators

Clase	Descripción
SchemaValidator	Contiene métodos para validar esquemas, asegurando que los campos requeridos estén presentes en los datos de entrada.

4.2.1.2. Interface Layer

Description of the design and components of the interface layer for the Identity and Access Context.

Schemas

Esquema	Descripción
UserSchemaGet	Esquema para la respuesta de la obtención de un usuario. Incluye <code>id</code> , <code>name</code> , <code>email</code> , <code>userType</code> , <code>image_url</code> , <code>registered</code> .
UserSchemaPost	Esquema para la creación de un nuevo usuario. Incluye <code>name</code> , <code>email</code> , <code>password</code> , <code>userType</code> .
UserChangeImage	Esquema para actualizar la imagen de un usuario. Incluye <code>image_url</code> , <code>role</code> .
VeterinarianSchemaPost	Esquema para la creación de un nuevo veterinario. Incluye <code>clinicName</code> , <code>otp_password</code> .
VeterinarianUpdateInformation	Esquema para la actualización de la información de un veterinario. Incluye <code>name</code> , <code>description</code> , <code>experience</code> .
VeterinarianSchemaGet	Esquema para la respuesta de la obtención de un veterinario. Incluye <code>id</code> , <code>name</code> , <code>clinicId</code> , <code>image_url</code> , <code>description</code> , <code>experience</code> , <code>user_id</code> .
VeterinarianProfileSchemaGet	Esquema para la respuesta detallada del perfil de un veterinario. Incluye <code>id</code> , <code>name</code> , <code>image_url</code> , <code>description</code> , <code>experience</code> , <code>clinicName</code> , <code>workingHourStart</code> , <code>workingHourEnd</code> , <code>clinicAddress</code> , <code>reviews</code> .

4.2.1.3. Application Layer

Description of the design and components of the application layer for the Identity and Access Context.

Services

Servicio	Método	Descripción
UserService	<code>get_user_by_id(user_id: int, db: Session)</code>	Recupera un usuario por su ID. Lanza una excepción 404 si el usuario no existe.
	<code>change_image(role_id: int, role, image: str, db: Session)</code>	Cambia la imagen de perfil del usuario según su rol (Owner o Veterinarian). Actualiza la entidad <code>User</code> y guarda los cambios.
PetOwnerService	<code>create_new_petowner(user_id: int, petowner: PetOwnerSchemaPost, db: Session = Depends(get_db))</code>	Crea un nuevo registro de propietario de mascotas. Verifica el tipo de usuario, si ya está registrado y el formato del teléfono. Emite un token de acceso.
	<code>get_petowners(db: Session = Depends(get_db))</code>	Recupera todos los propietarios de mascotas, incluyendo sus datos de usuario asociados.
	<code>get_petowner_by_user_id(user_id: int, db: Session)</code>	Recupera un propietario de mascotas por su ID de usuario.
	<code>get_petOwner_by_id(petOwner_id: int, db: Session) -> PetOwnerSchemaGet</code>	Recupera un propietario de mascotas por su ID.

Servicio	Método	Descripción
VeterinarianService	<code>change_Datapetowner(petowner_id: int, petowner: PetOwnerUpdateInformation, db: Session)</code>	Actualiza los datos de un propietario de mascotas existente.
	<code>create_new_veterinarian(user_id: int, veterinarian: VeterinarianSchemaPost, db: Session)</code>	Crea un nuevo registro de veterinario. Verifica el tipo de usuario, si ya está registrado, y valida el OTP y la clínica. Emite un token de acceso.
	<code>get_all_vets(db: Session = Depends(get_db)) -> List[VeterinarianSchemaGet]</code>	Recupera todos los veterinarios, incluyendo sus datos de usuario asociados.
	<code>get_vet_by_user_id(user_id: int, db: Session) -> VeterinarianSchemaGet</code>	Recupera un veterinario por su ID de usuario.
	<code>get_vet_by_id(vet_id: int, db: Session) -> VeterinarianSchemaGet</code>	Recupera un veterinario por su ID.
	<code>get_vet_by_id_details(vet_id: int, db: Session) -> VeterinarianProfileSchemaGet</code>	Recupera información detallada sobre un veterinario, incluyendo reseñas.
	<code>get_vets_by_clinic_id(clinic_id: int, db: Session) -> List[VeterinarianSchemaGet]</code>	Recupera veterinarios por ID de clínica.
	<code>get_available_times(vet_id: int, day: date, db: Session)</code>	Recupera los horarios disponibles para un veterinario en un día específico.
	<code>change_DataVet(vet_id: int, vet: VeterinarianUpdateInformation, db: Session)</code>	Actualiza los datos de un veterinario existente.

4.2.1.4. Infrastructure Layer

Description of the design and components of the infrastructure layer for the Identity and Access Context.\

Repositorios

Clase/Servicio	Descripción
UserRepository	Maneja la interacción con la base de datos para la entidad <code>User</code> . Incluye operaciones como crear, leer, actualizar y eliminar usuarios.
PetOwnerRepository	Maneja la interacción con la base de datos para la entidad <code>PetOwner</code> . Permite crear, leer, actualizar y eliminar propietarios de mascotas.
VeterinarianRepository	Maneja la interacción con la base de datos para la entidad <code>Veterinarian</code> . Incluye operaciones de gestión de veterinarios como crear, leer, actualizar y eliminar.

Mappers

Clase/Servicio	Descripción
UserMapper	Mapea la entidad <code>User</code> a la base de datos usando SQLAlchemy. Define cómo se traducen las propiedades del modelo <code>User</code> en columnas de la tabla de base de datos.

Clase/Servicio	Descripción
PetOwnerMapper	Mapea la entidad PetOwner a la base de datos usando SQLAlchemy. Define cómo se traducen las propiedades del modelo PetOwner en columnas de la tabla de base de datos.
VeterinarianMapper	Mapea la entidad Veterinarian a la base de datos usando SQLAlchemy. Define cómo se traducen las propiedades del modelo Veterinarian en columnas de la tabla de base de datos.

Routes

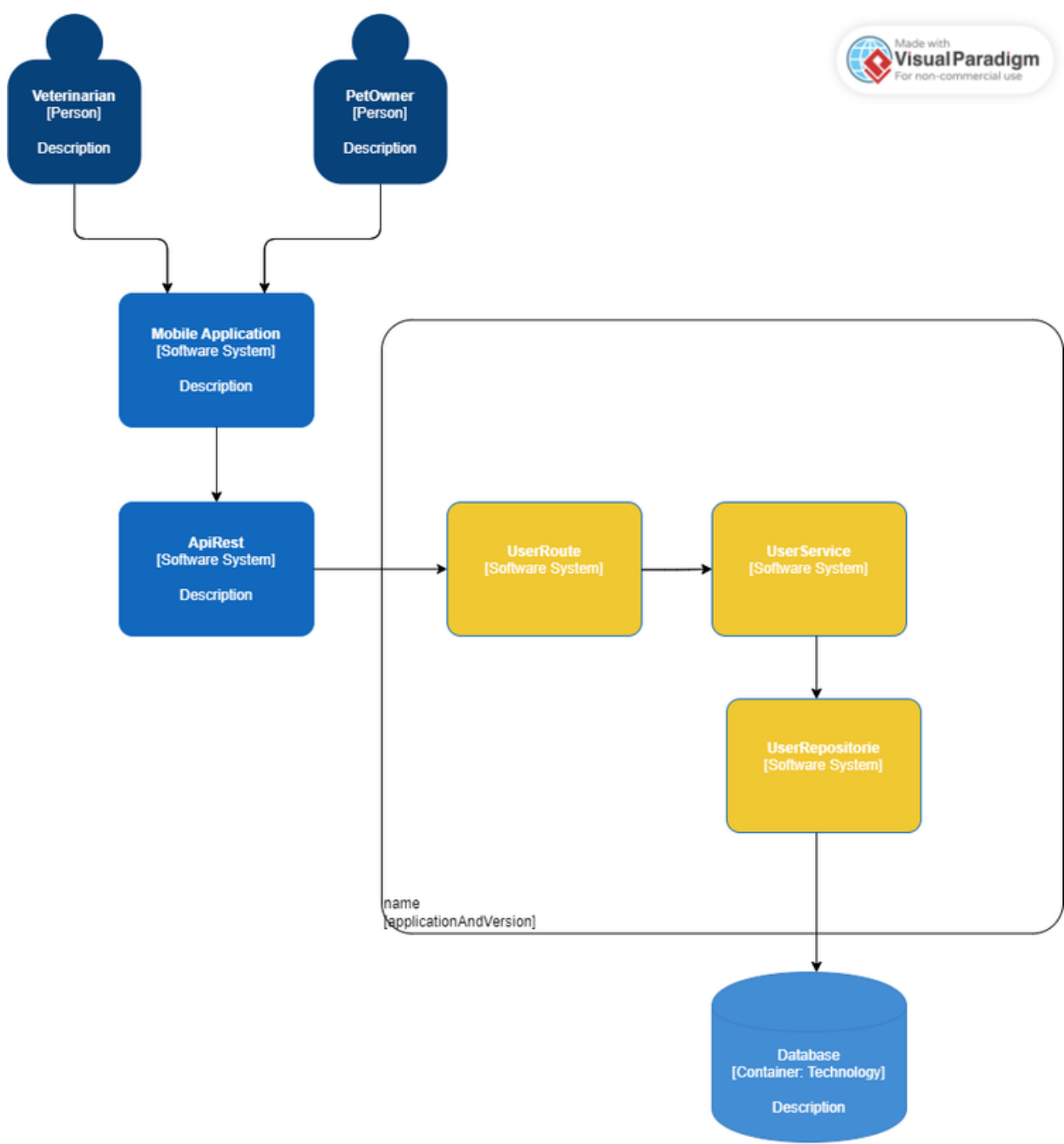
Ruta	Método	Descripción
PetOwners		
/petowners/{user_id}	POST	Crear un nuevo propietario de mascotas.
/petowners	GET	Obtener todos los propietarios de mascotas.
/petowners/users/{user_id}	GET	Obtener un propietario de mascotas por ID de usuario.
/petowners/{petOwner_id}	GET	Obtener un propietario de mascotas por ID de propietario.
/petowners/{petOwner_id}	PUT	Actualizar la información de un propietario de mascotas.
Users		
/users	GET	Obtener todos los usuarios.
/users/{user_id}	GET	Obtener un usuario por ID.
/users/{role_id}	PUT	Cambiar la imagen de un usuario.
Veterinarians		
/veterinarians/{user_id}	POST	Crear un nuevo veterinario.
/veterinarians	GET	Obtener todos los veterinarios.
/veterinarians/users/{user_id}	GET	Obtener un veterinario por ID de usuario.
/veterinarians/{vet_id}	GET	Obtener un veterinario por ID.
/veterinarians/vets/{clinic_id}	GET	Obtener veterinarios por ID de clínica.
/veterinarians/reviews/{vet_id}	GET	Obtener detalles del perfil de un veterinario, incluyendo reseñas.
/veterinarians/{vet_id}/available_times	POST	Obtener los horarios disponibles de un veterinario.
/veterinarians/{vet_id}	PUT	Actualizar la información de un veterinario.

4.2.1.5. Bounded Context Software Architecture Component Level Diagrams

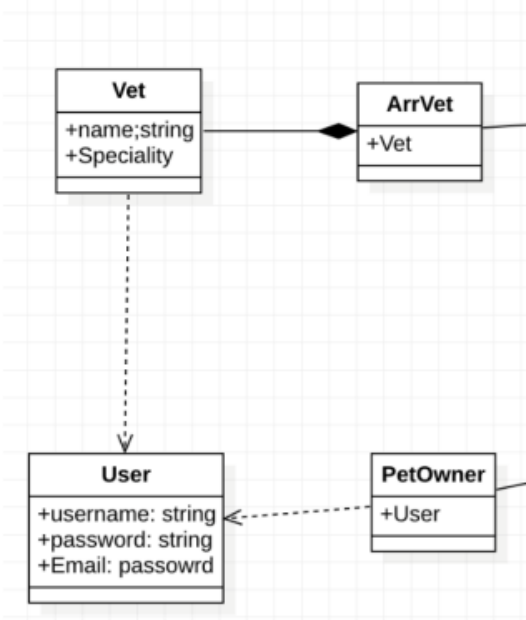
Component-level diagrams for the Identity and Access Context, showing the internal structure of components.

4.2.1.6. Bounded Context Software Architecture Code Level Diagrams

Code-level diagrams for the Identity and Access Context, detailing the classes and code structure.



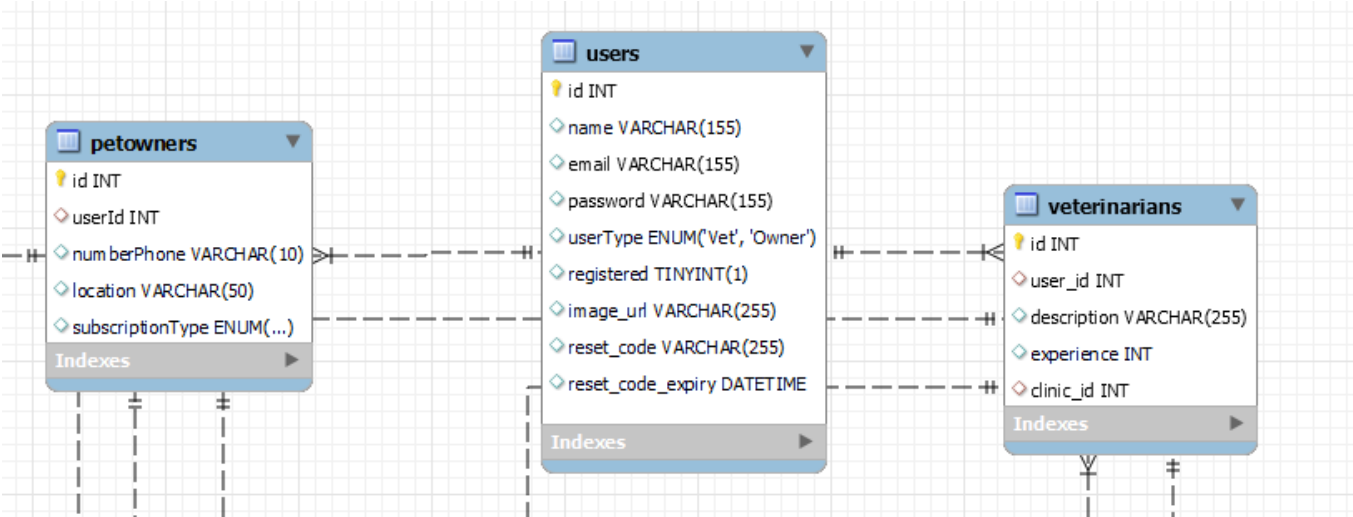
4.2.1.6.1. Bounded Context Domain Layer Class Diagrams



Class diagrams for the domain layer of the Identity and Access Context.

4.2.1.6.2. Bounded Context Database Design Diagram

Database design diagram for the Identity and Access Context.



4.2.2. Bounded Context: Veterinary Clinic Context

- [Domain Layer](#)
- [Interface Layer](#)
- [Application Layer](#)
- [Infrastructure Layer](#)
- [Bounded Context Software Architecture Component Level Diagrams](#)
- [Bounded Context Software Architecture Code Level Diagrams](#)

4.2.2.1. Domain Layer

Description of the design and components of the domain layer for the Veterinary Clinic Context.

Models

Clase	Descripción
OTP	Representa el código OTP (One Time Password) utilizado para verificar identidad. Atributos: <code>id</code> , <code>otp</code> , <code>expiration_time</code> , <code>clinicId</code> .

Clase	Descripción
VeterinaryClinic	Representa la entidad de una clínica veterinaria con atributos como <code>id</code> , <code>name</code> , <code>location</code> , <code>services</code> , <code>office_hours_start</code> , <code>office_hours_end</code> , <code>phone_number</code> , <code>description</code> , <code>image_url</code> . Relacionado con <code>Veterinarian</code> .
Availability	Representa la disponibilidad de un veterinario con atributos como <code>id</code> , <code>date</code> , <code>start_time</code> , <code>end_time</code> , <code>veterinarian_id</code> , <code>is_available</code> . Relacionado con <code>Veterinarian</code> .
Appointment	Representa una cita médica con atributos como <code>id</code> , <code>date_day</code> , <code>description</code> , <code>pet_id</code> , <code>veterinarian_id</code> , <code>start_time</code> , <code>end_time</code> , <code>status</code> , <code>diagnosis</code> , <code>treatment</code> . Relacionado con <code>Pet</code> y <code>Veterinarian</code> .

4.2.2.2. Interface Layer

Description of the design and components of the interface layer for the Veterinary Clinic Context.

Schemas

Esquema	Descripción
AppointmentSchemaCreate	Esquema para crear una nueva cita. Incluye <code>date_day</code> , <code>description</code> , <code>start_time</code> , <code>pet_id</code> , <code>veterinarian_id</code> . Método <code>to_model</code> para convertir a un modelo <code>Appointment</code> .
AppointmentSchemaUpdate	Esquema para actualizar una cita existente. Incluye <code>diagnosis</code> , <code>treatment</code> .
AppointmentSchemaGet	Esquema para obtener una cita. Incluye <code>id</code> , <code>date_day</code> , <code>description</code> , <code>start_time</code> , <code>end_time</code> , <code>pet_id</code> , <code>veterinarian_id</code> , <code>status</code> , <code>diagnosis</code> , <code>treatment</code> . Método <code>from_orm</code> para convertir de un modelo <code>Appointment</code> .
AvailabilitySchema	Esquema para disponibilidad de un veterinario. Incluye <code>date</code> , <code>start_time</code> , <code>end_time</code> , <code>veterinarian_id</code> , <code>is_available</code> . Método <code>from_orm</code> para convertir de un modelo <code>Availability</code> .
VeterinaryClinicSchemaPost	Esquema para la creación de una nueva clínica veterinaria. Incluye <code>name</code> , <code>location</code> , <code>phone_number</code> , <code>description</code> , <code>office_hours_start</code> , <code>office_hours_end</code> . Método <code>to_model</code> para convertir a un modelo <code>VeterinaryClinic</code> .
AvailabilitySchemaPost	Esquema para especificar la fecha de disponibilidad de un veterinario. Incluye <code>date</code> .
VeterinaryClinicSchemaGet	Esquema para obtener una clínica veterinaria. Incluye <code>id</code> , <code>name</code> , <code>location</code> , <code>services</code> , <code>image_url</code> , <code>description</code> , <code>phone_number</code> , <code>office_hours_start</code> , <code>office_hours_end</code> .

4.2.2.3. Application Layer

Description of the design and components of the application layer for the Veterinary Clinic Context.

Services

Servicio	Método	Descripción
AppointmentService	<code>get_all_appointments(db: Session)</code>	Recupera todas las citas y las devuelve como una lista de objetos <code>AppointmentSchemaGet</code> .
	<code>get_appointments_by_pet_id(pet_id: int, db: Session)</code>	Recupera todas las citas asociadas a una mascota por su ID. Lanza una excepción 404 si la mascota no existe.

Servicio	Método	Descripción
	<code>get_appointments_by_owner_id(owner_id: int, db: Session)</code>	Recupera todas las citas asociadas a un propietario de mascotas por su ID. Lanza una excepción 404 si el propietario no tiene mascotas registradas.
	<code>get_appointments(query, time_comparison_operator)</code>	Recupera citas basadas en un operador de comparación de tiempo. Ordena las citas en función del operador.
	<code>get_upcoming_appointments_by_owner_id(owner_id: int, db: Session)</code>	Recupera las citas próximas asociadas a un propietario de mascotas por su ID.
	<code>get_completed_appointments_by_owner_id(owner_id: int, db: Session)</code>	Recupera las citas completadas asociadas a un propietario de mascotas por su ID.
	<code>get_completed_appointments_by_veterinarian_id(veterinarian_id: int, db: Session)</code>	Recupera las citas completadas asociadas a un veterinario por su ID.
	<code>get_upcoming_appointments_by_veterinarian_id(veterinarian_id: int, db: Session)</code>	Recupera las citas próximas asociadas a un veterinario por su ID.
	<code>get_appointments_by_entity(entity, entity_id, db: Session, status_enum)</code>	Recupera citas basadas en una entidad (propietario o veterinario) y su estado. Lanza excepciones si la entidad no existe.
	<code>get_appointments_by_veterinarian_id(veterinarian_id: int, db: Session)</code>	Recupera todas las citas asociadas a un veterinario por su ID. Lanza una excepción 404 si el veterinario no existe.
	<code>get_appointment_by_id(appointment_id: int, db: Session)</code>	Recupera una cita específica por su ID. Lanza una excepción 404 si la cita no existe.
	<code>create_appointment(appointment: AppointmentSchemaCreate, db: Session)</code>	Crea una nueva cita y la guarda en la base de datos. Lanza excepciones si la mascota o el veterinario no existen.

Servicio	Método	Descripción
AvailabilityService	<code>post_appointment(appointment_id: int, result: AppointmentSchemaUpdate, db: Session)</code>	Actualiza una cita existente con nueva información y marca la cita como completada. Lanza una excepción 404 si la cita no existe.
	<code>check_and_reset_availabilities(db: Session)</code>	Verifica y restablece las disponibilidades semanales de los veterinarios.
	<code>create_availability(availability: AvailabilitySchema, db: Session)</code>	Crea una nueva disponibilidad y la guarda en la base de datos.
	<code>create_weekly_availabilities(db: Session)</code>	Crea disponibilidades semanales para todos los veterinarios en base a los horarios de su clínica.
	<code>create_weekly_by_new_veterinarian(vet: Veterinarian, db: Session)</code>	Crea disponibilidades semanales para un nuevo veterinario.
	<code>create_weekly_availabilities_for_veterinarian(vet: Veterinarian, db: Session, start_of_week: date, days_until_saturday: int)</code>	Crea disponibilidades semanales para un veterinario específico, desde el inicio hasta el final de la semana.
	<code>delete_weekly_availabilities(db: Session)</code>	Elimina todas las disponibilidades semanales existentes.
OTPServices	<code>generate_otp(clinic_id: int, db: Session)</code>	Genera un nuevo OTP para una clínica específica, válido por 10 minutos.
	<code>verify_otp(otp: str, db: Session)</code>	Verifica la validez de un OTP y devuelve el registro si es válido y no ha expirado.
	<code>delete_otp_record(otp_record: OTP, db: Session)</code>	Elimina un registro de OTP de la base de datos.
VeterinaryClinicService	<code>create_veterinary_clinic(clinic: VeterinaryClinicSchemaPost, db: Session)</code>	Crea una nueva clínica veterinaria y la guarda en la base de datos.
	<code>get_veterinary_clinics(db: Session)</code>	Recupera todas las clínicas veterinarias.
	<code>generate_unique_password(clinic_id: int, db: Session)</code>	Genera un OTP único para una clínica veterinaria específica.

Servicio	Método	Descripción
	<code>verify_veterinarian_register(clinic_name: str, otp_password: str, db: Session)</code>	Verifica la validez de un OTP para el registro de un veterinario, y compara el nombre de la clínica.
	<code>get_veterinary_clinic_by_id(clinic_id: int, db: Session)</code>	Recupera una clínica veterinaria específica por su ID.

4.2.2.4. Infrastructure Layer

Description of the design and components of the infrastructure layer for the Veterinary Clinic Context.

Routes

Ruta	Método	Descripción
Appointments		
<code>/appointments</code>	GET	Recupera todas las citas.
<code>/appointments/owner/{owner_id}</code>	GET	Recupera todas las citas asociadas a un propietario por su ID.
<code>/appointments/{appointment_id}</code>	GET	Recupera una cita específica por su ID.
<code>/appointments/pet/{pet_id}</code>	GET	Recupera todas las citas asociadas a una mascota por su ID.
<code>/appointments/veterinarian/{veterinarian_id}</code>	GET	Recupera todas las citas asociadas a un veterinario por su ID.
<code>/appointments</code>	POST	Crea una nueva cita.
<code>/appointments/owner/{owner_id}/upcoming</code>	GET	Recupera las citas próximas asociadas a un propietario por su ID.
<code>/appointments/owner/{owner_id}/past</code>	GET	Recupera las citas pasadas asociadas a un propietario por su ID.
<code>/appointments/veterinarian/{veterinarian_id}/upcoming</code>	GET	Recupera las citas próximas asociadas a un veterinario por su ID.
<code>/appointments/veterinarian/{veterinarian_id}/past</code>	GET	Recupera las citas pasadas asociadas a un veterinario por su ID.
<code>/appointments/{appointment_id}</code>	PUT	Actualiza una cita existente con nueva información.
Availabilities		
<code>/availabilities</code>	GET	Verifica y restablece las disponibilidades semanales de los veterinarios.
Veterinary Clinics		
<code>/veterinary_clinics</code>	POST	Crea una nueva clínica veterinaria.
<code>/veterinary_clinics</code>	GET	Recupera todas las clínicas veterinarias.
<code>/veterinary_clinics/generate_password/{clinic_id}</code>	GET	Genera una contraseña única para una clínica veterinaria.

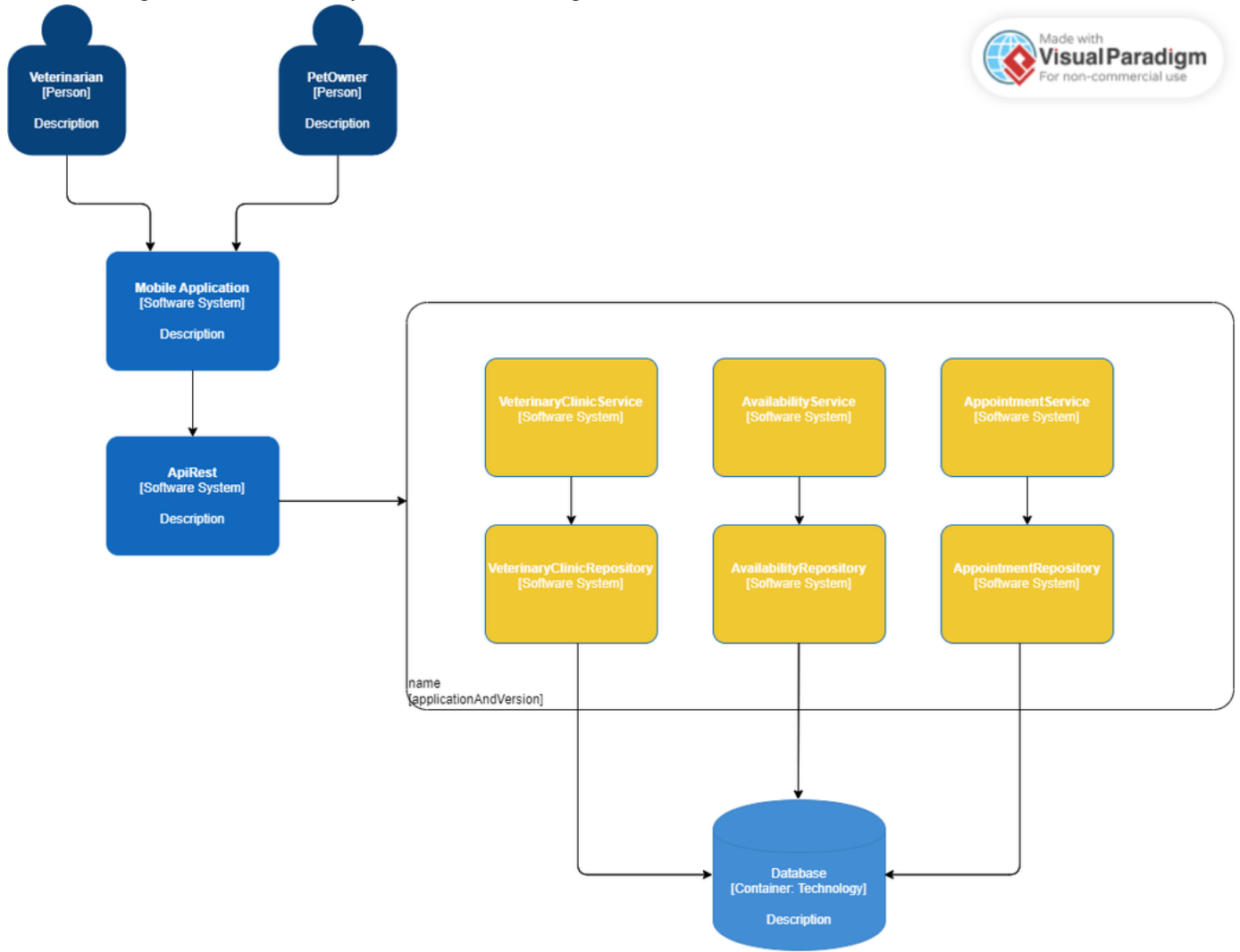
Ruta	Método	Descripción
/veterinary_clinics/{clinic_id}	GET	Recupera una clínica veterinaria específica por su ID.

4.2.2.5. Bounded Context Software Architecture Component Level Diagrams

Component-level diagrams for the Veterinary Clinic Context, showing the internal structure of components.

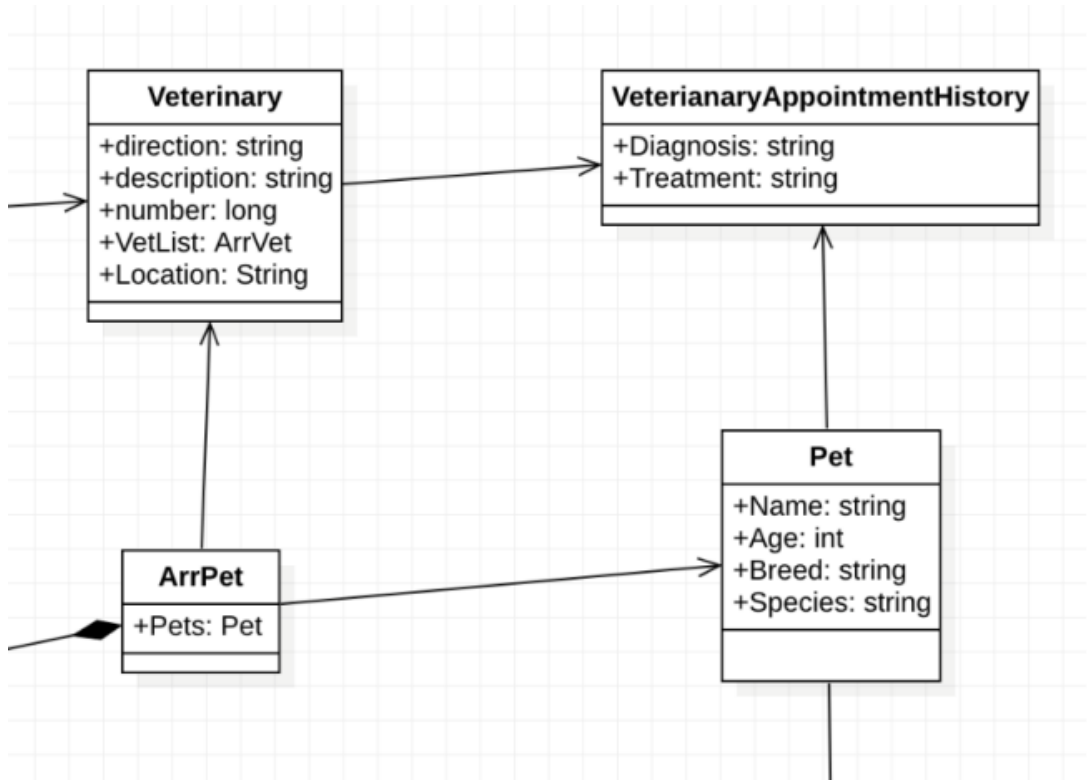
4.2.2.6. Bounded Context Software Architecture Code Level Diagrams

Code-level diagrams for the Veterinary Clinic Context, detailing the classes and code structure.



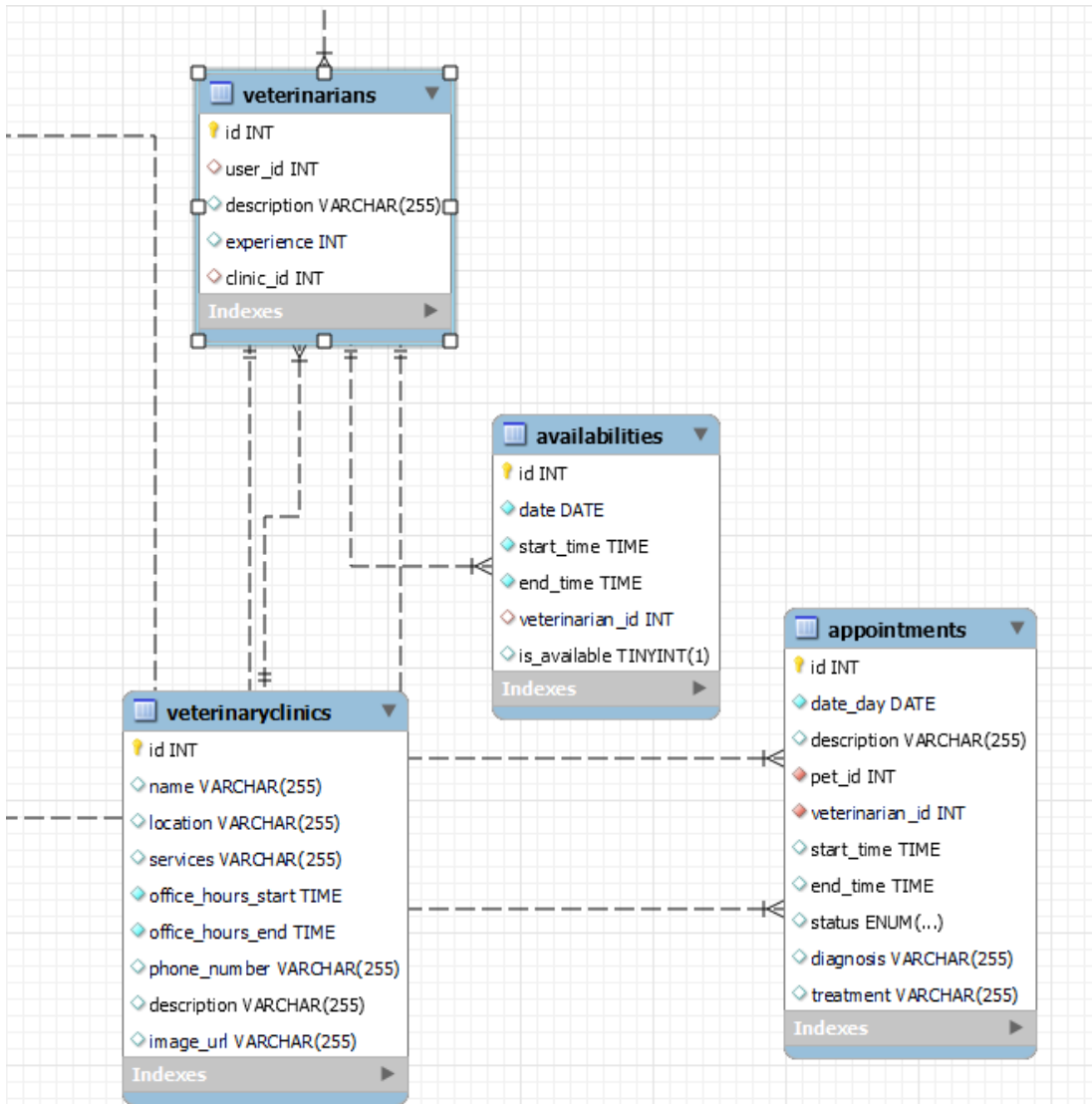
4.2.2.6.1. Bounded Context Domain Layer Class Diagrams

Class diagrams for the domain layer of the Veterinary Clinic Context.



4.2.2.6.2. Bounded Context Database Design Diagram

Database design diagram for the Veterinary Clinic Context.



4.2.3. Bounded Context: Medical Management Context

- [Domain Layer](#)
- [Interface Layer](#)
- [Application Layer](#)
- [Infrastructure Layer](#)
- [Bounded Context Software Architecture Component Level Diagrams](#)
- [Bounded Context Software Architecture Code Level Diagrams](#)

4.2.3.1. Domain Layer

Description of the design and components of the domain layer for the Medical Management Context.

Modelo	Descripción
Pet	Representa una mascota, incluyendo detalles como nombre, raza, especie, peso e historial médico. Tiene relaciones con <code>PetOwner</code> y <code>Appointment</code> .
PetOwner	Representa al propietario de una mascota. (Se asume que este modelo existe en <code>models.petOwner</code> , pero no se muestra en el código proporcionado)
MedicalHistory	Representa el historial médico de una mascota, incluyendo fechas, descripciones y relaciones con <code>Surgery</code> , <code>MedicalResult</code> , <code>Vaccine</code> y <code>Disease</code> .
Disease	Representa un diagnóstico de enfermedad dentro del historial médico de una mascota, incluyendo la fecha de diagnóstico, nombre y nivel de gravedad.

Modelo	Descripción
MedicalResult	Representa los resultados de pruebas médicas dentro del historial médico de una mascota, incluyendo la fecha del resultado, tipo y descripción.
Surgery	Representa un procedimiento quirúrgico dentro del historial médico de una mascota, incluyendo la fecha de la cirugía y la descripción.
Vaccine	Representa una vacuna administrada a una mascota, incluyendo el nombre de la vacuna, fecha, tipo, ubicación y dosis.

Nota: El modelo `PetOwner` se menciona en el modelo `Pet`, pero no se proporciona explícitamente. Se asume que está presente y definido adecuadamente en otra parte del código.

4.2.3.2. Interface Layer

Description of the design and components of the interface layer for the Medical Management Context.

Schemas

Esquema	Descripción
PetSchemaPost	Esquema para la creación de una mascota. Incluye campos como nombre, raza, especie, peso, fecha de nacimiento, URL de imagen y género.
PetSchemaResponse	Esquema para la respuesta que incluye todos los detalles de una mascota, como su ID, nombre, ID del propietario, raza, especie, peso, fecha de nacimiento, URL de imagen y género.
MedicalHistorySchemaPost	Esquema para la creación de un historial médico para una mascota. Incluye campos como ID de mascota, fecha y descripción.
MedicalHistorySchemaGet	Esquema para la obtención de un historial médico. Incluye ID, ID de mascota, fecha y descripción del historial médico.
DiseaseSchemaPost	Esquema para la creación de una enfermedad en el historial médico. Incluye nombre, ID del historial médico, fecha de diagnóstico y severidad.
DiseaseSchemaGet	Esquema para la obtención de una enfermedad del historial médico. Incluye ID, nombre, ID del historial médico, fecha de diagnóstico y severidad.
MedicalResultSchemaPost	Esquema para la creación de un resultado médico. Incluye fecha del resultado, tipo de resultado, descripción y ID del historial médico.
MedicalResultSchemaGet	Esquema para la obtención de un resultado médico. Incluye ID, fecha del resultado, tipo de resultado, descripción y ID del historial médico.
SurgerySchemaPost	Esquema para la creación de una cirugía en el historial médico. Incluye fecha de cirugía, descripción y ID del historial médico.
SurgerySchemaGet	Esquema para la obtención de una cirugía del historial médico. Incluye ID, fecha de cirugía, descripción y ID del historial médico.
VaccineSchemaPost	Esquema para la creación de una vacuna. Incluye nombre, fecha de vacuna, tipo de vacuna, dosis, ubicación y ID del historial médico.
VaccineSchemaGet	Esquema para la obtención de una vacuna. Incluye ID, nombre, fecha de vacuna, tipo de vacuna, dosis, ubicación y ID del historial médico.

Cada esquema está diseñado para transformar los datos en objetos Python y validar los datos entrantes y salientes de acuerdo con los requisitos del modelo de dominio.

4.2.3.3. Application Layer

Description of the design and components of the application layer for the Medical Management Context.

Services

Servicio	Método	Descripción
DiseaseService	<code>add_disease(medical_history_id: int, disease: DiseaseSchemaPost, db: Session)</code>	Añade una nueva enfermedad al historial médico. Valida la existencia del historial médico y el esquema de la enfermedad.
	<code>get_all_diseases_by_medical_history_id(medical_history_id: int, db: Session)</code>	Recupera todas las enfermedades asociadas a un historial médico específico. Lanza una excepción 404 si no se encuentran.
MedicalResultService	<code>add_medical_result(medical_history_id: int, medical_result: MedicalResultSchemaPost, db: Session)</code>	Añade un nuevo resultado médico al historial médico. Valida la existencia del historial médico y el esquema del resultado.
	<code>get_all_medical_results_by_medical_history_id(medical_history_id: int, db: Session)</code>	Recupera todos los resultados médicos asociados a un historial médico específico. Lanza una excepción 404 si no se encuentran.
SurgeryService	<code>add_surgery(medical_history_id: int, surgery: SurgerySchemaPost, db: Session)</code>	Añade una nueva cirugía al historial médico. Valida la existencia del historial médico y el esquema de la cirugía.
	<code>get_all_surgeries_by_medical_history_id(medical_history_id: int, db: Session)</code>	Recupera todas las cirugías asociadas a un historial médico específico. Lanza una excepción 404 si no se encuentran.

Servicio	Método	Descripción
VaccineService	<code>add_vaccine(medical_history_id: int, vaccine: VaccineSchemaPost, db: Session)</code>	Añade una nueva vacuna al historial médico. Valida la existencia del historial médico y el esquema de la vacuna.
	<code>get_all_vaccines_by_medical_history_id(medical_history_id: int, db: Session)</code>	Recupera todas las vacunas asociadas a un historial médico específico. Lanza una excepción 404 si no se encuentran.
PetServices	<code>create_new_pet(petowner_id: int, pet: PetSchemaPost, db: Session)</code>	Crea un nuevo registro de mascota. Verifica la existencia del propietario y valida los datos de la mascota.
	<code>get_pet_by_user_id(pet_id: int, db: Session)</code>	Recupera una mascota por su ID.
	<code>get_pets_by_petOwnerid(petOwner_id: int, db: Session)</code>	Recupera todas las mascotas asociadas a un propietario específico.
	<code>update_pet(pet_id: int, pet: PetSchemaPost, db: Session)</code>	Actualiza los datos de una mascota existente.
	<code>get_pet_by_id(pet_id: int, db: Session)</code>	Recupera una mascota por su ID.
	<code>delete_pet(pet_id: int, db: Session)</code>	Elimina una mascota por su ID.
	<code>add_medical_history(medical_history: MedicalHistorySchemaPost, db: Session)</code>	Añade un nuevo historial médico.
MedicalHistoryService	<code>get_all_medical_histories(db: Session)</code>	Recupera todos los historiales médicos.
	<code>get_medical_history_by_pet_id(pet_id: int, db: Session)</code>	Recupera el historial médico asociado a una mascota por su ID.
	<code>get_medical_history(medical_history_id: int, db: Session)</code>	Recupera un historial médico por su ID.

Servicio	Método	Descripción
	<code>add_medical_result(medical_history_id: int, medical_result: MedicalResultSchemaPost, db: Session)</code>	Añade un resultado médico a un historial médico.
	<code>add_disease(medical_history_id: int, disease: DiseaseSchemaPost, db: Session)</code>	Añade una enfermedad a un historial médico.
	<code>add_surgery(medical_history_id: int, surgery: SurgerySchemaPost, db: Session)</code>	Añade una cirugía a un historial médico.
	<code>add_vaccine(medical_history_id: int, vaccine: VaccineSchemaPost, db: Session)</code>	Añade una vacuna a un historial médico.
	<code>get_all_medical_results_by_id(medical_history_id: int, db: Session)</code>	Recupera todos los resultados médicos asociados a un historial médico específico.
	<code>get_all_diseases_by_id(medical_history_id: int, db: Session)</code>	Recupera todas las enfermedades asociadas a un historial médico específico.
	<code>get_all_surgeries_by_id(medical_history_id: int, db: Session)</code>	Recupera todas las cirugías asociadas a un historial médico específico.
	<code>get_all_vaccines_by_id(medical_history_id: int, db: Session)</code>	Recupera todas las vacunas asociadas a un historial médico específico.

4.2.3.4. Infrastructure Layer

Description of the design and components of the infrastructure layer for the Medical Management Context.

Ruta	Método	Descripción
Diseases		
<code>/diseases</code>		Endpoint principal para el servicio de enfermedades.
MedicalHistories		
<code>/medicalhistories</code>	POST	Crea un nuevo historial médico.
<code>/medicalhistories</code>	GET	Recupera todos los historiales médicos.
<code>/medicalhistories/pet/{pet_id}</code>	GET	Recupera el historial médico asociado a una mascota por su ID.
<code>/medicalhistories/{medical_history_id}</code>	GET	Recupera un historial médico por su ID.

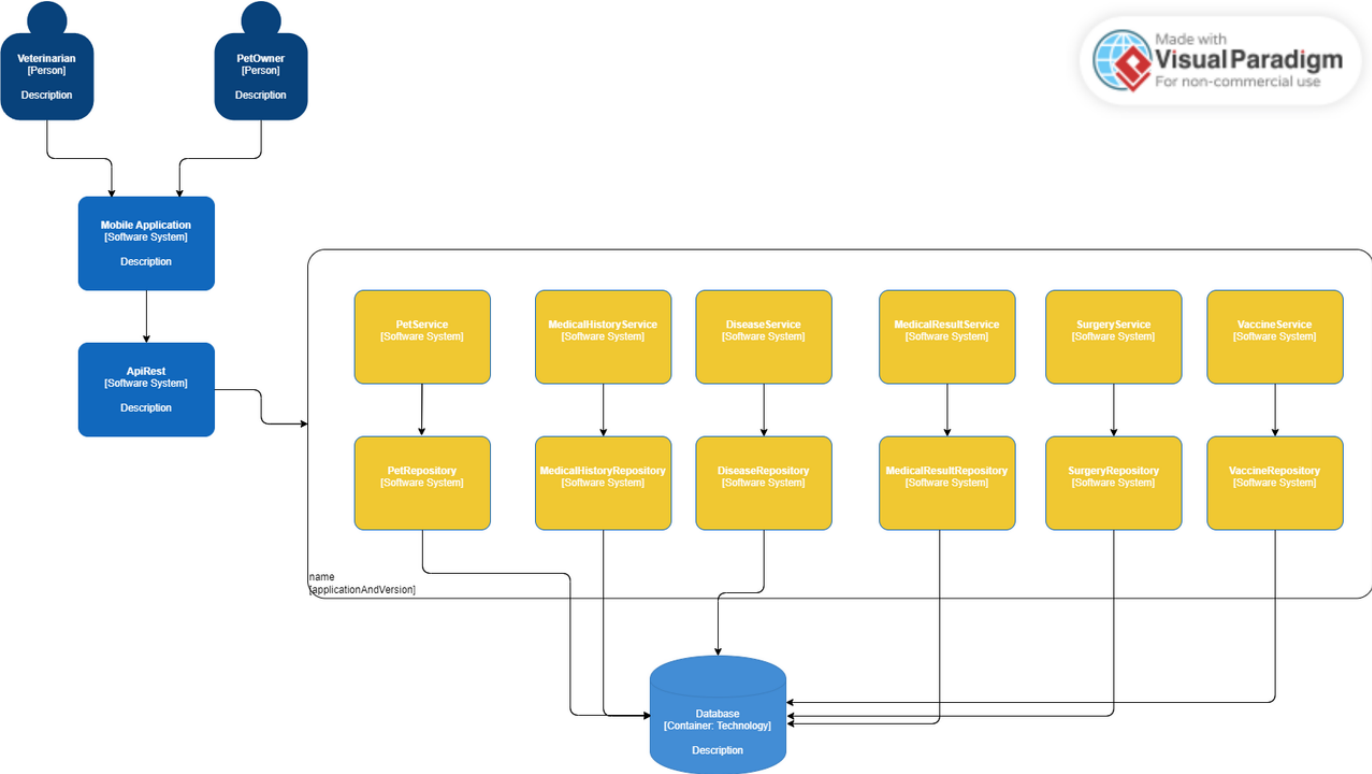
Ruta	Método	Descripción
/medicalhistories/{medical_history_id}/medicalresults	POST	Añade un resultado médico a un historial médico.
/medicalhistories/{medical_history_id}/diseases	POST	Añade una enfermedad a un historial médico.
/medicalhistories/{medical_history_id}/surgeries	POST	Añade una cirugía a un historial médico.
/medicalhistories/{medical_history_id}/vaccines	POST	Añade una vacuna a un historial médico.
/medicalhistories/{medical_history_id}/medicalresults	GET	Recupera todos los resultados médicos asociados a un historial médico específico.
/medicalhistories/{medical_history_id}/diseases	GET	Recupera todas las enfermedades asociadas a un historial médico específico.
/medicalhistories/{medical_history_id}/surgeries	GET	Recupera todas las cirugías asociadas a un historial médico específico.
/medicalhistories/{medical_history_id}/vaccines	GET	Recupera todas las vacunas asociadas a un historial médico específico.
Pets		
/pets/{petowner_id}	POST	Crea una nueva mascota asociada a un propietario.
/pets	GET	Recupera todas las mascotas.
/pets/{petowner_id}	GET	Recupera todas las mascotas asociadas a un propietario específico.
/pets/{pet_id}	PUT	Actualiza los datos de una mascota existente.
/pets/pet/{pet_id}	GET	Recupera una mascota por su ID.
/pets/{pet_id}	DELETE	Elimina una mascota por su ID.
Vaccinations		
/vaccinations		Endpoint principal para el servicio de vacunas.

4.2.3.5. Bounded Context Software Architecture Component Level Diagrams

Component-level diagrams for the Medical Management Context, showing the internal structure of components.

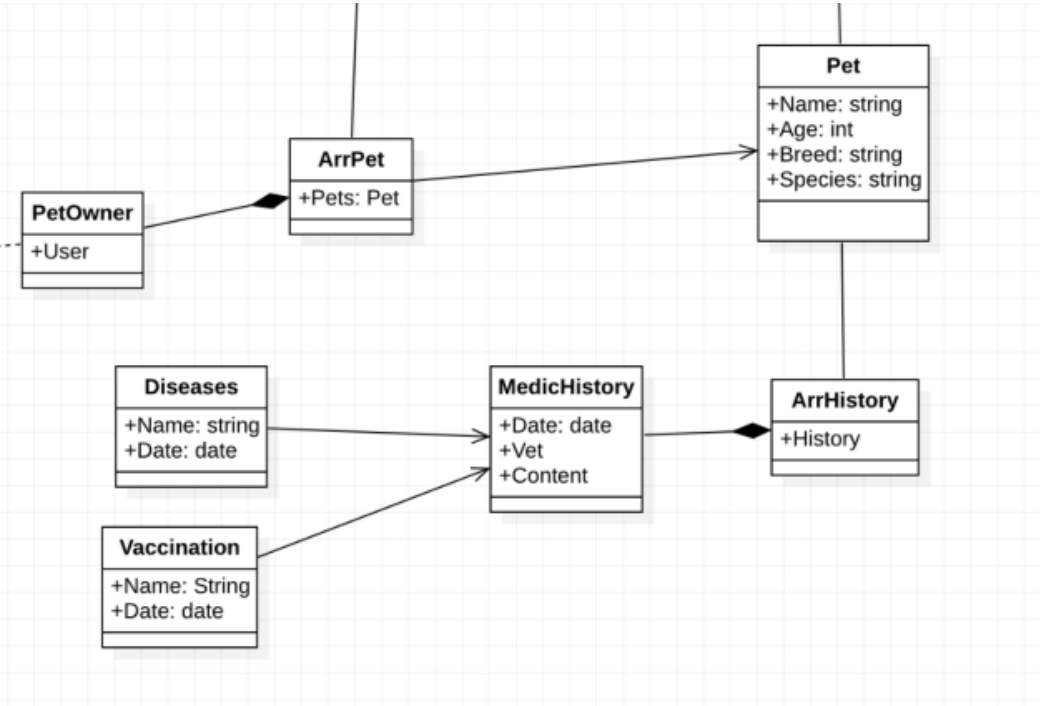
4.2.3.6. Bounded Context Software Architecture Code Level Diagrams

Code-level diagrams for the Medical Management Context, detailing the classes and code structure.



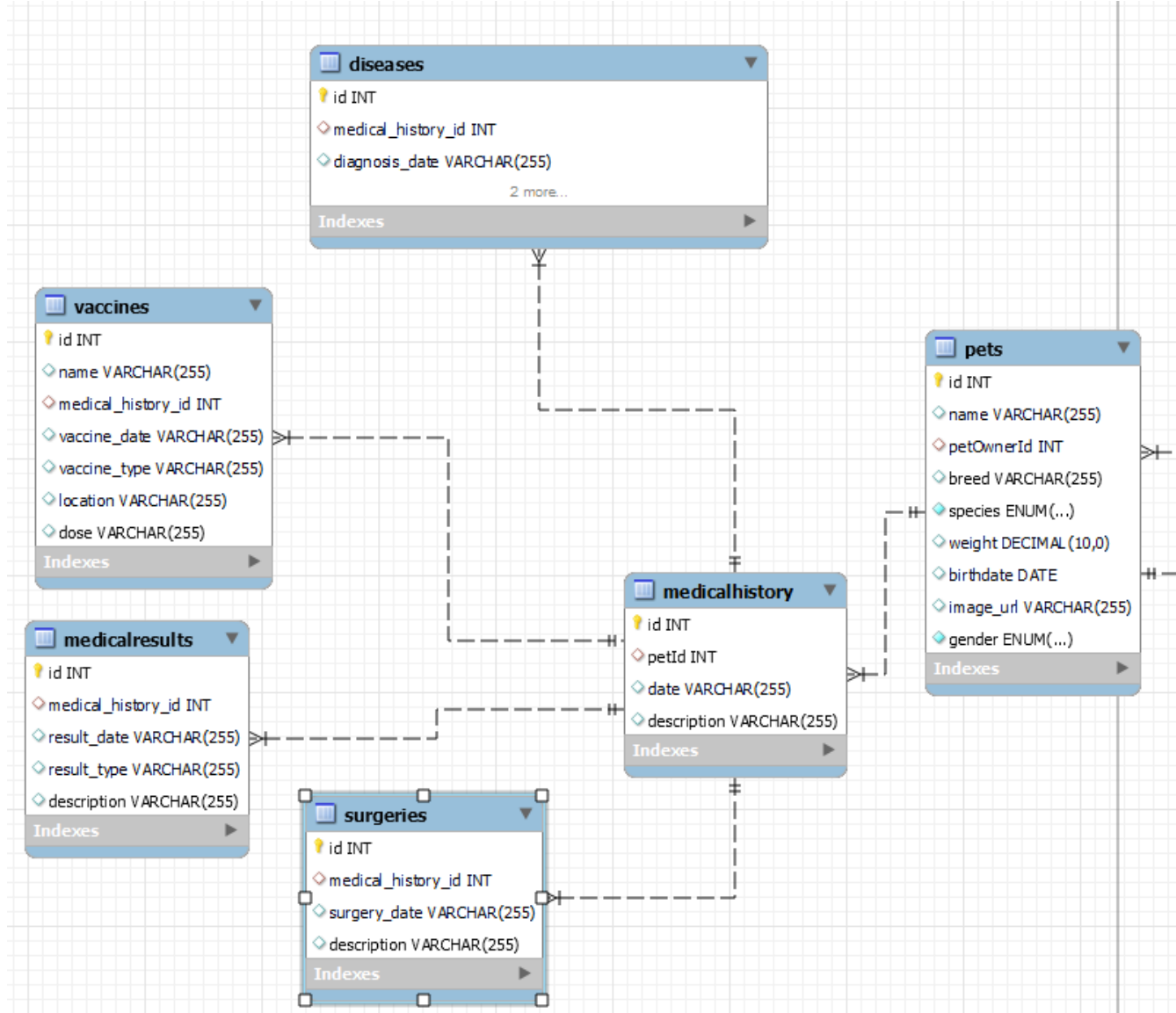
4.2.3.6.1. Bounded Context Domain Layer Class Diagrams

Class diagrams for the domain layer of the Medical Management Context.



4.2.3.6.2. Bounded Context Database Design Diagram

Database design diagram for the Medical Management Context.



4.2.4. Bounded Context: User Experience Context

- [Domain Layer](#)
- [Interface Layer](#)
- [Application Layer](#)
- [Infrastructure Layer](#)
- [Bounded Context Software Architecture Component Level Diagrams](#)
- [Bounded Context Software Architecture Code Level Diagrams](#)

4.2.4.1. Domain Layer

Description of the design and components of the domain layer for the User Experience Context.

Models

Clase	Descripción
Notification	Representa una notificación para un propietario de mascotas. Atributos: <code>id</code> , <code>petOwnerId</code> , <code>type</code> , <code>message</code> , <code>datetime</code> . Relacionado con <code>PetOwner</code> .
Review	Representa una reseña de un veterinario realizada por un propietario de mascotas. Atributos: <code>id</code> , <code>stars</code> , <code>description</code> , <code>review_time</code> , <code>petowner_id</code> , <code>veterinarian_id</code> . Relacionado con <code>PetOwner</code> y <code>Veterinarian</code> .

4.2.4.2. Interface Layer

Description of the design and components of the interface layer for the User Experience Context.

Schemas

Esquema	Descripción
NotificationSchemaPost	Esquema para la creación de una notificación. Atributos: <code>petOwnerId</code> , <code>type</code> , <code>message</code> , <code>datetime</code> .
NotificationSchemaGet	Esquema para la obtención de una notificación. Atributos: <code>id</code> , <code>petOwnerId</code> , <code>type</code> , <code>message</code> , <code>datetime</code> .
ReviewSchemaPost	Esquema para la creación de una reseña. Atributos: <code>description</code> , <code>stars</code> , <code>veterinarian_id</code> . Método <code>to_model</code> para convertir el esquema en una instancia de <code>Review</code> .
ReviewSchemaGet	Esquema para la obtención de una reseña. Atributos: <code>id</code> , <code>description</code> , <code>stars</code> , <code>review_time</code> , <code>image_url</code> , <code>pet_owner_name</code> . Método <code>from_orm</code> para convertir una instancia de <code>Review</code> en el esquema.

4.2.4.3. Application Layer

Description of the design and components of the application layer for the User Experience Context.

Services

Servicio	Método	Descripción
ReviewService	<code>create_new_review(petowner_id: int, review: ReviewSchemaPost, db: Session)</code>	Crea una nueva reseña. Valida que el propietario de la mascota exista, luego crea y guarda la reseña en la base de datos. Devuelve la reseña creada como <code>ReviewSchemaGet</code> .
	<code>get_all_reviews(db: Session)</code>	Recupera todas las reseñas. Incluye las relaciones con <code>PetOwner</code> y <code>User</code> , y devuelve una lista de reseñas como <code>ReviewSchemaGet</code> .
	<code>get_reviews_by_veterinarian_id(vet_id: int, db: Session)</code>	Obtiene todas las reseñas asociadas a un veterinario específico. Filtra por <code>veterinarian_id</code> , incluye relaciones con <code>PetOwner</code> y <code>User</code> , y devuelve una lista de reseñas como <code>ReviewSchemaGet</code> .

4.2.4.4. Infrastructure Layer

Description of the design and components of the infrastructure layer for the User Experience Context.

Routes

Ruta	Método	Descripción
Notifications		
<code>/notifications</code>	POST	Crea una nueva notificación. Valida la existencia del propietario de la mascota antes de guardar la notificación. Devuelve la notificación creada como <code>NotificationSchemaGet</code> .
<code>/notifications</code>	GET	Obtiene todas las notificaciones almacenadas en la base de datos. Devuelve una lista de notificaciones como <code>NotificationSchemaGet</code> .
<code>/notifications/petowner/{petowner_id}</code>	GET	Obtiene todas las notificaciones asociadas a un propietario de mascota específico. Valida la existencia del propietario y devuelve las notificaciones correspondientes.
Reviews		

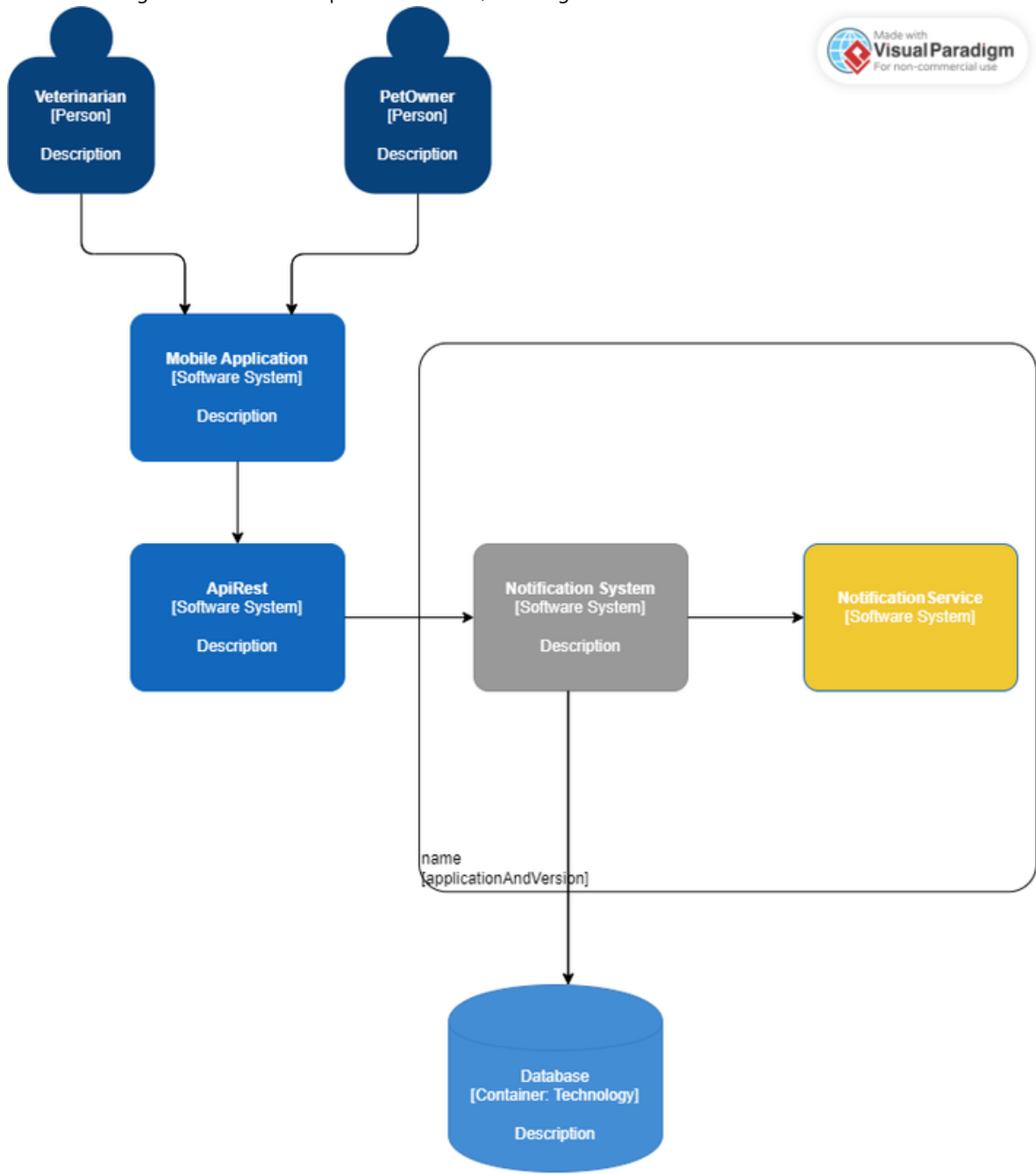
Ruta	Método	Descripción
/reviews	POST	Crea una nueva reseña para un propietario de mascota específico. Utiliza <code>ReviewService</code> para validar y guardar la reseña. Devuelve la reseña creada como <code>ReviewSchemaGet</code> .
/reviews	GET	Obtiene todas las reseñas almacenadas en la base de datos. Utiliza <code>ReviewService</code> para recuperar y devolver una lista de reseñas como <code>ReviewSchemaGet</code> .

4.2.4.5. Bounded Context Software Architecture Component Level Diagrams

Component-level diagrams for the User Experience Context, showing the internal structure of components.

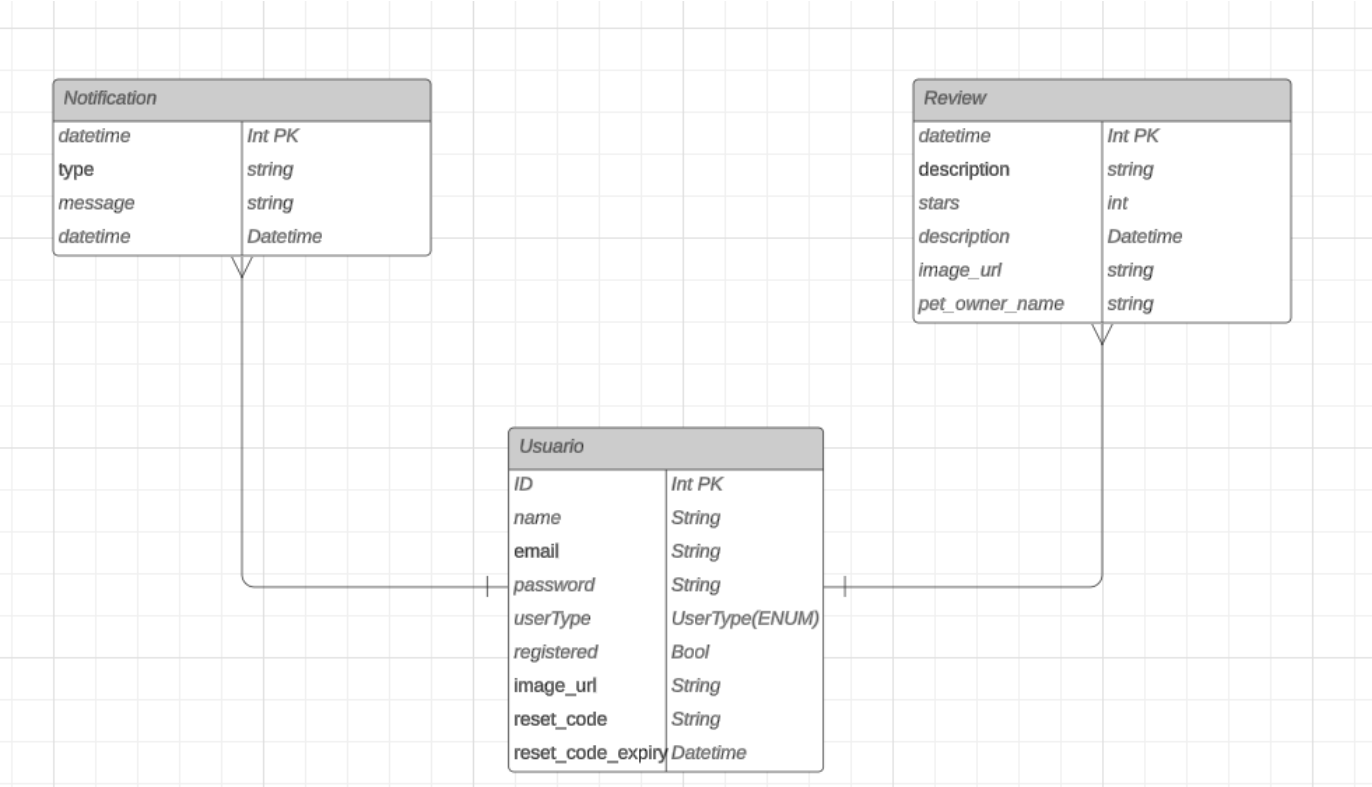
4.2.4.6. Bounded Context Software Architecture Code Level Diagrams

Code-level diagrams for the User Experience Context, detailing the classes and code structure.



4.2.4.6.1. Bounded Context Domain Layer Class Diagrams

Class diagrams for the domain layer of the User Experience Context.



4.2.4.6.2. Bounded Context Database Design Diagram

Database design diagram for the User Experience Context.

